

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке метода и программной реализации сервиса оценки «справедливой» цены аренды и покупки квартир в городе.

Работа направлена на повышение обоснованности ценовых решений и снижение асимметрии информации между участниками рынка. Подход основывается на моделях машинного обучения для табличных данных с учётом геопространственных признаков, пространственно-временной валидации, построения доверительных интервалов и интерпретации факторов влияния.

Результатом являются методика, сервис и экспериментальная оценка на реальных данных: прогноз цены с доверительным интервалом, объяснения вклада признаков и подбор сопоставимых объектов, подтверждающие практическую применимость решения.

Работа состоит из 61 страниц, включает в себя 9 рисунков и 5 таблиц.

Ключевые слова: интеллектуальная система анализа недвижимости, оценка стоимости недвижимости, аренда квартир, машинное обучение, геопространственные признаки, прогнозирование стоимости, доверительный интервал, интерпретация модели, SHAP, компараблы, REST API, веб-сервис.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ТЕРМИНОВ	7
ВВЕДЕНИЕ	10
ЛИТЕРАТУРНЫЙ ОБЗОР	13
1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ	14
1.1 Анализ предметной области	14
1.2 Обзор существующих аналогов.....	15
1.3 Цель и задачи	17
1.4 Техническое задание	18
1.4.1 Назначение и цели разработки системы.....	18
1.4.2 Функциональные требования	19
1.4.3 Нефункциональные требования	19
1.4.4 Требования к аппаратно-программной платформе	20
1.4.4.1 Аппаратная часть	20
1.4.4.2 Программная часть.....	20
1.4.5 Порядок контроля	20
1.4.6 Требования к программной документации	21
1.4.7 Стадии и этапы разработки	21
1.5 Вывод по исследовательскому разделу	22
2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	23
2.1 Выбор языка и инструментов программирования.....	23
2.2 Обоснование выбора методов прогнозирования и компонентов решения...	24
2.3 Модель бизнес-процесса в нотации IDEF0	28
2.4 Анализ архитектуры системы.....	32
2.5 Вывод по аналитическому разделу	35
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	37
3.1 Подготовка данных для обучения модели.....	37
3.2 Обучение нейронной сети	39
3.3 Описание программной реализации	42

3.3.1	ML-сервис	43
3.3.2	Backend-шлюз.....	45
3.3.3	Веб-интерфейс	47
3.4	Тестирование приложения	49
3.5	Расчёт надёжности	52
3.6	Руководство пользователя.....	54
3.6.1	Доступ и требования:.....	54
3.6.2	Основные варианты использования	55
3.6.2.1	Расчёт стоимости по адресу	55
3.6.2.2	Проверка статуса сервиса.....	55
3.6.3	Порядок работы.....	56
3.6.4	Рекомендации по корректному вводу данных	56
3.6.5	Получение отчёта	56
3.6.6	Политика данных	57
3.7	Вывод по технологическому разделу	57
	ЗАКЛЮЧЕНИЕ	58
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	61
	ПРИЛОЖЕНИЯ.....	65

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ТЕРМИНОВ

AI (Artificial Intelligence) – искусственный интеллект; в работе используется для обозначения модуля генерации текстовых комментариев к оценке.

API (Application Programming Interface) – программный интерфейс приложения; набор HTTP-эндпоинтов, через которые фронтенд и внешние системы обращаются к сервису оценки.

AVM (Automated Valuation Model) – автоматизированная модель оценки недвижимости, рассчитывающая стоимость объекта на основе статистических и/или ML-моделей без участия оценщика.

НЗ – иерархический шестигранный геопространственный индекс, применяемый для разбиения территории на ячейки фиксированного масштаба.

JSON (JavaScript Object Notation) – текстовый формат обмена структурированными данными, используемый в запросах и ответах API.

MAE (Mean Absolute Error) – средняя абсолютная ошибка; метрика качества модели регрессии, измеряющая среднее по модулю отклонение прогноза от фактического значения.

MAPE (Mean Absolute Percentage Error) – средняя абсолютная процентная ошибка; метрика качества регрессии, показывающая среднее относительное отклонение прогноза в процентах.

ML (Machine Learning) – машинное обучение; набор методов построения моделей на основе данных без явного программирования правил.

PDF (Portable Document Format) – формат электронных документов; в работе используется для генерации отчётов об оценке.

REST (Representational State Transfer) – архитектурный стиль веб-сервисов; в работе обозначает HTTP-API backend-шлюза.

RMSE (Root Mean Squared Error) – корень из средней квадратичной ошибки; метрика качества регрессии, более чувствительная к крупным ошибкам.

SHAP (SHapley Additive exPlanations) – метод интерпретации моделей машинного обучения на основе значений Шепли; в работе используется для построения локальных объяснений вклада признаков.

UI (User Interface) – пользовательский интерфейс; веб-страница RentAdvisor для ввода параметров квартиры и просмотра результатов.

БД – база данных; централизованное хранилище объявлений, геопризнаков и результатов оценок.

Блок-кросс-валидация – вариант кросс-валидации, при котором наблюдения группируются в пространственные блоки (ячейки НЗ/районы), исключая «подсматривание» в соседние локации.

Витрина признаков – согласованное хранилище подготовленных признаков и целевых переменных, используемое для обучения и инференса моделей.

Геопризнаки – геопропространственные признаки объекта (индекс НЗ, расстояние до метро, характеристики района и др.), используемые моделью.

ГИС – геоинформационные системы; программные средства для работы с пространственными данными и картографией.

Доверительный интервал – диапазон значений, в котором с заданной доверительной вероятностью ожидается «справедливая» арендная ставка.

Инференс – этап применения обученной модели для расчёта прогноза по новым объектам.

Компараблы (сопоставимые объекты) – выбранные по схожести объявлений квартиры, используемые для проверки и обоснования рассчитанной цены.

Квантильная регрессия – вариант регрессионной модели, напрямую оценивающей заданные квантили распределения целевой переменной (нижнюю и верхнюю границы интервала).

Конформное предсказание – метод построения предсказательных интервалов с гарантированным уровнем покрытия без жёстких предположений о распределении ошибок.

Логирование (логгер) – механизм регистрации событий работы системы (запросы, ошибки, предупреждения, время выполнения операций) в виде журналов (логов) для диагностики, мониторинга и анализа работы сервисов.

Медианный прогноз – точечная оценка стоимости, соответствующая медиане условного распределения арендной ставки для объекта.

Пространственно-временная валидация – стратегия оценки модели, при которой данные разделяются одновременно по времени и по пространственным блокам для честной проверки переносимости.

ВВЕДЕНИЕ

Российский рынок жилья демонстрирует выраженную динамичность цен и различия по сегментам/локациям; официальные статистики фиксируют существенные колебания индексов цен и сдвиги в ипотечной активности, влияющие на доступность жилья и арендные ставки Центральный Банк России [1]. Профильные обзоры указывают на дисбалансы спроса и предложения на рынке аренды и ускорение арендных ставок в отдельные периоды [2], что повышает неопределённость для домохозяйств и профессиональных участников. В этих условиях система, объединяющая геопространственные признаки, интерпретируемые модели и честную оценку неопределённости, имеет практическую ценность для обоснования сделок и снижения риска ошибок.

Для реализации интеллектуальной системы анализа недвижимости была выбрана каскадная модель жизненного цикла программного обеспечения. Ее применение обусловлено тем, что разработка системы предполагает последовательное выполнение взаимосвязанных этапов: анализ предметной области и требований к оценке стоимости аренды квартир, проектирование архитектуры сервиса, подготовку и обработку данных, разработку моделей машинного обучения, реализацию модулей прогнозирования, построения доверительного интервала, интерпретации факторов и подбора сопоставимых объектов, интеграцию backend-шлюза и веб-интерфейса, а также проведение тестирования корректности и надежности работы системы. Такой подход позволяет обеспечить управляемость процесса разработки, контроль результатов каждого этапа и соответствие программного продукта поставленным задачам выпускной квалификационной работы.

Объект исследования – рынок жилой недвижимости и модели предиктивной аналитики; предмет – прогнозирование стоимости с использованием геопространственных и объектных характеристик.

В качестве данных используются объявления о недвижимости (тип сделки, цена, параметры объекта, адрес), геопризнаки местоположения (транспортная доступность, близость инфраструктуры), а также временной контекст публикации [3] [4] [5]. Выполнены очистка, геокодирование и агрегация признаков по пространственным ячейкам района.

Методически работа опирается на современные модели для табличных данных (включая градиентный бустинг над решениям деревьев), пространственно-временную валидацию (разделение по времени и блочную кросс-валидацию по районам), построение доверительных интервалов (квантильная регрессия/конформные предсказания) и методы интерпретации вкладов признаков (SHAP). Для демонстрации практической полезности реализован подбор «компараблов» на основе близости по ключевым признакам и локализации.

Практическая значимость заключается в повышении прозрачности и обоснованности ценовых решений на рынке, а также в возможности интеграции в веб-сервис оценки стоимости с картографической визуализацией и объяснениями.

В Приложении А представлено организационно-экономическое обоснование выпускной квалификационной работы.

В Приложении Б представлен графический материал (презентация) выпускной квалификационной работы.

В процессе написания выпускной квалификационной работы руководствовался следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ (с изменениями, в действующей редакции с 26.11.2024) [2].
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ (с изменениями на 31.07.2025) [3].

3. «О гражданской обороне» от 12.02.1998 № 28-ФЗ (с изменениями на 23.07.2025) [4].
4. Приказ Минздравсоцразвития РФ от 03.05.2024 № 220н «Об утверждении Порядка оказания первой помощи» [5].
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (с изменениями на 09.02.2026) [6].
6. Санитарные правила СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда» [7].
7. СанПиН 1.1.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания» [8].

ЛИТЕРАТУРНЫЙ ОБЗОР

В настоящей работе использованы и проанализированы публикации, отражающие как состояние рынка жилья, так и современные подходы к его аналитике. Официальные отчёты Банка России и ДОМ.РФ фиксируют рост нагрузки на домохозяйства, неравномерную динамику цен и дефицит качественного арендного жилья, что задаёт контекст задачи «справедливой» оценки арендной ставки [1] [Ошибка! Источник ссылки не найден.]. Работы по массовой оценке недвижимости и анализу ценового поведения квартир выделяют ключевые факторы стоимости (локация, характеристики дома и квартиры, окружение) и ограничения традиционных гедонических моделей [8] [11].

Отдельный блок источников посвящён применению методов машинного обучения в оценке и анализе рынка недвижимости. Публикации по использованию ML-моделей для оценки коммерческой и жилой недвижимости демонстрируют преимущества деревьев решений и градиентного бустинга по сравнению с классическими регрессиями, а также поднимают вопросы интерпретируемости и устойчивости моделей [9] [10].

Методическую основу подхода к моделированию составили исследования по пространственно-временной валидации и блок-кросс-валидации для структурированных данных [12] [13], работы по конформным предсказаниям и квантильной регрессии для построения доверительных интервалов [14] [15] [16], а также публикации по интерпретируемости моделей на основе SHAP-значений и XAI в табличных задачах [21] [22] [23]. Совокупно эти источники определили выбор архитектуры решения, набора метрик и требований к объяснимости прогноза.

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

В исследовательском разделе рассматривается постановка задачи, область применения и существующие подходы к оценке стоимости жилой недвижимости. Выполнены анализ предметной области и аналогов, по результатам которого сформулированы цель и задачи работы и подготовлено техническое задание с основными требованиями к системе.

1.1 Анализ предметной области

В рамках работы рассматривается предметная область оценки рыночной стоимости квартир для аренды с использованием методов предиктивной аналитики и геоинформационных систем. Она лежит на пересечении рынков жилья, прикладной аналитики табличных и геопространственных данных и методов машинного обучения. Объектом оценки выступают квартиры в аренду; ключевые характеристики: локация (адрес, координаты), параметры планировки (площадь, комнаты, этаж/этажность), тип и состояние дома, окружение (доступность транспорта, объекты инфраструктуры), а также временной контекст публикации.

Существующие подходы включают:

- эконометрические гедонические модели (линейная/лог-линейная регрессия): прозрачны и интерпретируемы, но плохо захватывают нелинейности и взаимодействия признаков;
- правила/эвристики оценщиков и СМА (сравнительный анализ рынка): понятны экспертам, но трудоёмки, субъективны и слабо масштабируемы [8] [9] [10] [11].

Предлагаемое решение опирается на современные табличные модели с расширенным геопризнаковым описанием (агрегаты по ячейкам местности, время/стоимость до транспорта, плотность и структура предложений), честную

оценку неопределённости (квантильные/конформные предсказания), интерпретируемость (локальные и глобальные объяснения вкладов признаков) и поиск компараблов по близости атрибутов и локации. Такой подход призван объединить масштабируемость автоматизированных моделей с прозрачностью и практической полезностью экспертных методик, что создаёт основу для последующей постановки целей, задач и формализации технического задания.

1.2 Обзор существующих аналогов

В рамках обзора аналогов были рассмотрены наиболее известные платформы российского рынка недвижимости, предоставляющие пользователю ориентировочную оценку объекта: Яндекс.Недвижимость и ЦИАН. Сравнение проводилось по следующим критериям:

- наличие оценки по адресу;
- доверительного интервала;
- объяснения факторов;
- подбора похожих объектов;
- возможности экспорта результата;
- прозрачности метода;
- публичного API и ориентации на прикладной пользовательский сценарий.

Яндекс.Недвижимость предоставляет пользователю ориентировочную оценку стоимости объекта по адресу и параметрам квартиры, а также позволяет просматривать похожие предложения. К сильным сторонам платформы можно отнести удобный интерфейс, широкое покрытие объявлений и ориентацию на практический сценарий выбора или уточнения параметров объекта. Однако к недостаткам относятся отсутствие доверительного интервала, отсутствие объяснения факторов, влияющих на итоговую оценку, закрытость методики расчёта, а также отсутствие средств экспорта результата и публичного API.

Таким образом, сервис удобен как инструмент получения быстрого ориентира, но не обеспечивает достаточной прозрачности и аналитической глубины результата.

ЦИАН также позволяет получить оценку объекта по адресу и параметрам квартиры, а кроме того, предоставляет пользователю диапазон стоимости и подбор похожих объектов. Это делает платформу более полезной с точки зрения первичного анализа рынка по сравнению с рядом других решений. К сильным сторонам ЦИАН относятся наглядность результата, наличие диапазона оценки и использование большой базы рыночных предложений. В то же время сервис, как и Яндекс.Недвижимость, не раскрывает метод расчёта в достаточной степени, не показывает объяснение вклада факторов, не предоставляет экспорт результата и не имеет открытого публичного API. Следовательно, несмотря на более информативный вывод, ЦИАН остаётся в первую очередь пользовательским сервисом оценки, а не прозрачным аналитическим инструментом.

Разрабатываемая система отличается от рассмотренных аналогов тем, что помимо оценки по адресу включает доверительный интервал, объяснение факторов, подбор сопоставимых объектов, формирование отчёта, прозрачность результата за счёт интерпретации модели и наличие REST API для программного доступа. Тем самым система ориентирована не только на получение числовой оценки, но и на аналитическое сопровождение результата. Это особенно важно в условиях высокой неопределённости рынка недвижимости, когда пользователю необходимо понимать не только итоговое значение, но и причины его формирования.

По результатам сравнительного анализа можно сделать следующие выводы.

1. Яндекс.Недвижимость и ЦИАН решают задачу быстрой ориентировочной оценки стоимости объекта и обладают сильной стороной в виде доступа к большим массивам рыночных данных.

2. ЦИАН по сравнению с Яндекс.Недвижимостью предоставляет несколько более информативный результат за счёт отображения диапазона стоимости.
3. Общими недостатками рассмотренных аналогов являются слабая прозрачность метода, отсутствие объяснения факторов, отсутствие полноценного экспорта результатов и закрытость программного доступа.
4. Разрабатываемая система направлена на устранение указанных ограничений за счёт включения интерпретируемой модели, оценки неопределённости, механизма подбора компараблов и API-доступа.

1.3 Цель и задачи

Цель работы – построить модель, предоставляющую прогноз с доверительным интервалом и подбором компараблов на основе методов машинного обучения и геопространственных признаков.

Для достижения поставленной цели необходимо провести анализ предметной области и обзор существующих аналогов, сформировать и утвердить техническое задание. На следующем этапе требуется собрать датасет из объявлений и внешних геопризнаков, выполнить его очистку, геокодирование и генерацию признаков. Затем разрабатываются и обучаются модели прогнозирования с настройкой пространственно-временной валидации.

Особое внимание уделяется реализации оценки неопределённости (квантильной или конформной) и созданию модуля интерпретации вкладов признаков. Дополнительно предусматривается механизм поиска сопоставимых объектов («компараблов») по атрибутам и локализации. Завершающим этапом является создание веб-сервиса с картографической визуализацией и возможностью формирования краткого отчёта, а также проведение

экспериментальной оценки качества и подготовка пользовательской и эксплуатационной документации.

1.4 Техническое задание

В разделе представлено техническое задание на разработку системы «Интеллектуальная система анализа недвижимости». Техническое задание разработано в соответствии с ГОСТ 34.602-2020 [6].

1.4.1 Назначение и цели разработки системы

Полное наименование темы: «Интеллектуальная система анализа недвижимости».

Назначение: автоматизированная оценка «справедливой» стоимости аренды квартир по объектным и геопространственным признакам с предоставлением объяснений и сопоставимых объектов для поддержки принятия решений.

Цель работы: разработать систему, формирующий прогноз цены с доверительным интервалом, интерпретируемым объяснением вкладов факторов и подбором компараблов, а также предоставляющий краткий отчёт пользователю.

Задачи работы:

1. Выполнить анализ предметной области и обзор аналогов.
2. Сформировать и утвердить техническое задание.
3. Сформировать датасет объявлений и внешних геопризнаков; выполнить очистку и геокодирование.
4. Спроектировать и обучить модели прогнозирования с пространственно-временной валидацией.

5. Реализовать оценку неопределённости (квантильная/конформная) и интерпретацию вкладов признаков.
6. Реализовать подбор сопоставимых объектов.
7. Создать веб-сервис с картографической визуализацией и экспортом отчёта.
8. Провести экспериментальную оценку качества и подготовить документацию.

1.4.2 Функциональные требования

1. Ввод данных об объекте: адрес/координаты, тип сделки (аренда), площадь(и), количество комнат, этаж/этажность, тип/состояние дома и др.
2. Геокодирование и валидация ввода: определение координат по адресу; проверка корректности и полноты полей.
3. Прогноз стоимости: расчёт «справедливой» цены аренды; вывод доверительного интервала.
4. Интерпретация результата: отображение локальных вкладов признаков (объяснение для конкретного объекта) и краткого глобального резюме факторов модели.
5. Подбор компараблов: выдача 5–10 сопоставимых объектов с указанием расстояния/времени до транспорта и ключевых атрибутов.
6. Отчёт пользователю: формирование краткого отчёта (PDF/HTML) с ценой, интервалом, объяснениями и списком компараблов.
7. Локализация: интерфейс и отчёт на русском языке.

1.4.3 Нефункциональные требования

- время формирования прогноза – до 2 секунд при типовом запросе; генерация отчёта – до 5 секунд;

- точность предсказания должна быть выше 75%;

1.4.4 Требования к аппаратно-программной платформе

Приведенные требования к аппаратной и программной части должны гарантировать корректную работу программного обеспечения.

1.4.4.1 Аппаратная часть

Для запуска программного обеспечения и его корректной работы рекомендуется использовать позиции со следующими минимальными требованиями:

- сервер: 4 vCPU, 8 ГБ ОЗУ, SSD 100 ГБ; сетевое подключение 100 Мбит/с и выше;
- рабочая станция администратора: 4 ядра CPU, 8 ГБ ОЗУ, SSD 128 ГБ.

1.4.4.2 Программная часть

1. Операционная система: Windows 10 и выше/Linux.
2. РСУБД: PostgreSQL.
3. Python 3.10 и выше.
4. Компоненты веб-сервиса
5. Подсистемы: сервис карт и геокодирования; средство генерации отчётов; система журналирования и мониторинга.

1.4.5 Порядок контроля

В ходе разработки необходимо обеспечивать контроль работоспособности системы, фиксировать все обнаруженные неисправности и выполнять их устранение.

1.4.6 Требования к программной документации

Состав документации:

- руководство пользователя – содержит назначение системы, требования к программно-технической среде, описание основных вариантов использования (расчёт стоимости по адресу, проверка статуса сервиса), порядок работы с веб-интерфейсом, рекомендации по корректному;
- пояснительную записку – описание предметной области и постановки задачи, технического задания, архитектуры и процессов системы, методов подготовки данных и моделирования, результатов тестирования и расчёта надёжности;
- спецификацию API (машиночитаемую Swagger/OpenAPI-спецификацию) – перечень REST-эндпоинтов, структур запросов и ответов, кодов ошибок и используемых форматов.

1.4.7 Стадии и этапы разработки

Согласно ГОСТ Р ИСО/МЭК 12207–2010 [7] и ГОСТ 34.602-2020 [6] выделяются стадии:

1. Техническое задание: подготовка и утверждение ТЗ.
2. Эскизный/технический проект: архитектура, модели данных/признаков, проект интерфейса.
3. Рабочий проект (реализация): разработка модулей, интеграция, наполнение данными, обучение моделей.
4. Внедрение и испытания: тестирование, калибровка, приёмка, подготовка документации.

Этап подготовительных работ включает анализ предметной области и существующих аналогов, формирование требований и рисков, а также подготовку и утверждение технического задания.

Этап работы с данными предусматривает сбор, очистку и геокодирование информации, построение геопризнаков и проведение контроля качества полученных данных.

Этап моделирования охватывает выбор базовой и целевой моделей, настройку пространственно-временной валидации, обучение моделей, оценку неопределённости прогнозов и интерпретацию вкладов признаков.

Этап разработки прикладных модулей направлена на реализацию функций подбора сопоставимых объектов, автоматическую генерацию отчётов, а также разработку средств картографической визуализации.

Этап интеграции и развертывания включает создание и настройку серверной части, реализацию систем логирования и мониторинга, а также обеспечение мер информационной безопасности.

Заключительный этап испытаний и приёмки предусматривает проведение функциональных, нефункциональных и пользовательских тестов, подготовку отчёта о соответствии системы техническому заданию и оформление итоговой документации.

1.5 Вывод по исследовательскому разделу

Проведено исследование задачи и предметной области, сформулированы базовые термины и требования к данным и моделям. На основе анализа были выделены ключевые функциональные дефициты существующих подходов – недостаток прозрачности, отсутствие доверительных интервалов и системного подбора сопоставимых объектов; их компенсация положена в основу требований к разрабатываемой системе (интервальная оценка, интерпретируемость, компараблы). На этой базе сформулированы цель и задачи, а также подготовлено компактное техническое задание, определяющее функционал, нефункциональные характеристики и контуры дальнейшего проектирования и реализации.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

В данном разделе подробно описываются используемые методы прогнозирования стоимости аренды и принципы их работы: модели машинного обучения для табличных данных с учётом геопространственных признаков, подходы к получению интервальных предсказаний и интерпретации влияния признаков. Обосновывается выбор инструментов реализации, а также приводится архитектура программного решения и схема взаимодействия его основных компонентов. Здесь же вводятся метрики качества (в том числе MAE, MAPE, RMSE по логарифму ставки и покрытие доверительных интервалов), по которым в дальнейшем оценивается выполнение требования к точности системы не ниже 75%.

2.1 Выбор языка и инструментов программирования

Основным языком выбран Python [24] – является стандартным для анализа данных и ML благодаря простому синтаксису, широкой экосистеме библиотек и зрелым инструментам воспроизводимых экспериментов. Это позволяет быстро строить прототипы моделей, автоматизировать обработку данных и интегрировать результаты в веб-сервис.

Инструменты обработки данных и геоаналитики:

- pandas / NumPy – табличные данные и численные расчёты;
- shapely – геометрии, проекции, пространственные операции;
- h3-py – иерархическая дискретизация пространства.

Инструменты моделирования и объяснимости:

- scikit-learn – базовые модели/валидация/пайплайны;
- LightGBM/CatBoost/XGBoost – градиентный бустинг над деревьями решений для табличных данных [18] [19] [20]; поддержка квантильной регрессии;

- SHAP – локальные и глобальные интерпретации вкладов признаков;
- MAPIE – построение доверительных интервалов поверх произвольной модели.

Инфраструктура экспериментов и воспроизводимость:

5. **MLflow** – учёт экспериментов, метрик и артефактов, «реестр» моделей.
6. **Poetry / pip + venv** – управление зависимостями.
7. **Docker** – контейнеризация среды инференса и вспомогательных сервисов.

Сервисная часть и отчётность:

8. **FastAPI** [26] – REST API сервис.
9. **PostgreSQL + PostGIS** [27] – хранилище табличных и пространственных данных.
10. **Parquet** – компактное колоночное хранилище витрин/фичей.
11. **WeasyPrint / ReportLab / Jinja2** – генерация HTML/PDF-отчётов.
12. **Folium / Leaflet/JS** – картографическая визуализация объекта и компараблов.

Выбранный стек минимизирует время до работающего сервиса и покрывает ключевые требования: геообогащение, точный табличный ML, интерпретация, доверительные интервалы, компараблы и веб-интеграция.

2.2 Обоснование выбора методов прогнозирования и компонентов решения

В рамках задачи оценки «справедливой» стоимости объектов недвижимости критично учитывать три особенности предметной области:

- сильную неоднородность признаков (категориальные, числовые, геопространственные, календарные);

- выраженную пространственно-временную зависимость цен (локальные рынки и сезонность);
- необходимость практического результата с доверительным интервалом и прозрачным объяснением факторов.

Исходя из этого, архитектура аналитической части решения строится вокруг градиентного бустинга над деревьям решений и дополняется модулями неопределённости, интерпретации и отбора сопоставимых объектов.

Практическое разделение ролей:

- **CatBoost** – предпочтителен при большом числе категориальных полей и умеренном размере датасета: даёт стабильные результаты «из коробки» (ordered boosting, встроенное кодирование категорий);
- **LightGBM** – оптимален, когда важна скорость обучения/инференса и необходимо быстро перебирать конфигурации (histogram-based, leaf-wise рост дерева);
- **XGBoost** – надёжен как «референс» и бенчмарк, особенно на табличных задачах с аккуратно подготовленными признаками.

Для повышения качества применяется байесовская оптимизация гиперпараметров (Optuna) с единой целевой метрикой (например, MAE по лог-цене) и контрольными ограничениями по стабильности на отложенных временных и пространственных блоках.

Система должна не только выдавать точку прогноза, но и честно показывать диапазон неопределённости. В решении предусмотрены два комплементарных механизма:

- **квантильная регрессия** в бустинге: обучение отдельных моделей (или единой многоквантильной) на τ -квантили (например, 0.1/0.5/0.9) даёт «быстрые» пессимистичные/медианные/оптимистичные оценки;
- **конформные интервалы** поверх базовой модели: калибровка ширины интервала на валидационном наборе с гарантией покрытий при слабых предположениях об ошибках [14] [15] [16] [17].

На практике это даёт пользователю и эксперту рынка ясное представление о колебании цены и уровне риска.

Для объяснения прогноза применяются SHAP-значения:

- **локально** (для конкретного объекта) показывается вклад каждого признака: площадь, шаговая доступность, удалённость от метро, этажность, состояние;
- **глобально** агрегируются важности и зависимость эффекта от признака (dependence plots), что формирует краткое «резюме факторов» для отчёта [21] [22] [23];

Такой формат прозрачен для арендаторов, покупателей и специалистов рынка, а также помогает выявлять артефакты данных.

Модуль компараблов поддерживает двухступенчатый отбор:

1. **Строгая фильтрация** по близости локации, типу сделки, классу/типу дома, количеству комнат, диапазонам площади и этажности, давности объявления.
2. **Ранжирование** по составной метрике схожести (весовая комбинация нормированных отличий по ключевым полям) с географическим штрафом за расстояние/время в пути до ядра локации.

Для географической части используется **индексация на сетке НЗ** (или аналог) для физически корректных пространственных окон; расстояния считаются по **Haversine**, а при наличии источников — дополнительно оценивается **время до транспорта** (общественный транспорт/магистраль) и «эффект барьеров» (реки/железные дороги) через penalization.

Чтобы избежать утечки и завышения метрик:

- **временное разбиение** (train на прошлых периодах, следовательно test на будущих) проверяет устойчивость к смене сезона и конъюнктуры;
- **spatial blocking** (блокировка по НЗ-ячейкам) минимизирует «подсматривание» в соседние улицы/кварталы [12] [13];

- итоговая оценка – **сводная** (time split × spatial blocks) и сопровождается доверительными интервалами метрик (bootstrap/перестановки).

В потоке данных встречаются нетипичные объекты (лофты, таунхаусы на границе города, элитные ЖК). Для повышения надёжности:

- применяется **детектор OOD** (например, Isolation Forest или расстояние до обучающего многообразия в пространстве признаков);
- отчёт содержит **предупреждения** о снижении доверия и рекомендации по ручной проверке/уточнению входных параметров.

Конфигурируемый пайплайн: ingest → clean → гео-обогащение → генерация признаков → сплиты → обучение → калибровка неопределённости → отчёты.

Все шаги описываются YAML-конфигами, команды – единым CLI на Typer. Это обеспечивает воспроизводимость экспериментов и переносимость сценариев.

Для последующей интеграции с бэкендом на Go [25]:

- экспортируются артефакты модели и REST API-контракты для инференса;
- формируются **PDF отчёты** для пользователя (стоимость, интервал, объяснения, 5–10 компараблов);
- поддерживается легковесный HTTP-сервис инференса с возможностью горизонтального масштабирования (Gin framework).

Ограничения и меры снижения риска:

- **смещение данных**, рынок меняется – предусмотрено **переобучение по расписанию** и мониторинг с дрейф-метриками;
- **неполные адреса/геокодирование**, строена валидация ввода и fallback-процедуры; отчёт явно помечает низкое доверие;
- **редкие классы объектов**, для малых сегментов включаются техники **reweighting/upsampling** и расширяется окно компараблов.

Такой набор методов даёт практическое покрытие требований: точечный и интервальный прогноз, прозрачные объяснения, сопоставимые объекты «рядом и похожие», проверенная на разрывах в пространстве и времени валидация – при этом весь стек совместим с дальнейшей веб-интеграцией и промышленным развёртыванием. Стил ь изложения и уровень детализации соответствуют образцам: акцент на прикладной логике и архитектурных решениях без углубления в математические выкладки.

2.3 Модель бизнес-процесса в нотации IDEF0

Для наглядного представления функциональности интеллектуальной системы анализа недвижимости выполнено моделирование бизнес-процесса в нотации IDEF0. Диаграммы отражают входы/выходы, управляющие воздействия и механизмы, обеспечивающие работу процесса, а также декомпозицию основных функций на более детальные подпроцессы.

На контекстном уровне представлен процесс использования веб-сервиса оценки «справедливой» стоимости, который преобразует ввод пользователя (адрес и параметры квартиры) и базовые внешние данные о локации в прогноз цены с доверительным интервалом, объяснения факторов и список сопоставимых объектов, формируя итоговый отчёт.

На вход подаются параметры квартиры (адрес/координаты, площадь, комнаты, этаж и др.), управляющие потоки: правила валидации данных, конфигурация модели и пороги качества (схема признаков, уровень интервала и др.) и техническое задание, в качестве механизмов, включают веб-интерфейс, сервис инференса (ML-модель), базовые геоданные/БД и генератор отчётов. На выход получаютс я цена (точка) и доверительный интервал, краткие объяснения, компараблы и отчёт (PDF/HTML). Контекстный уровень представлен на Рисунке 2.1.

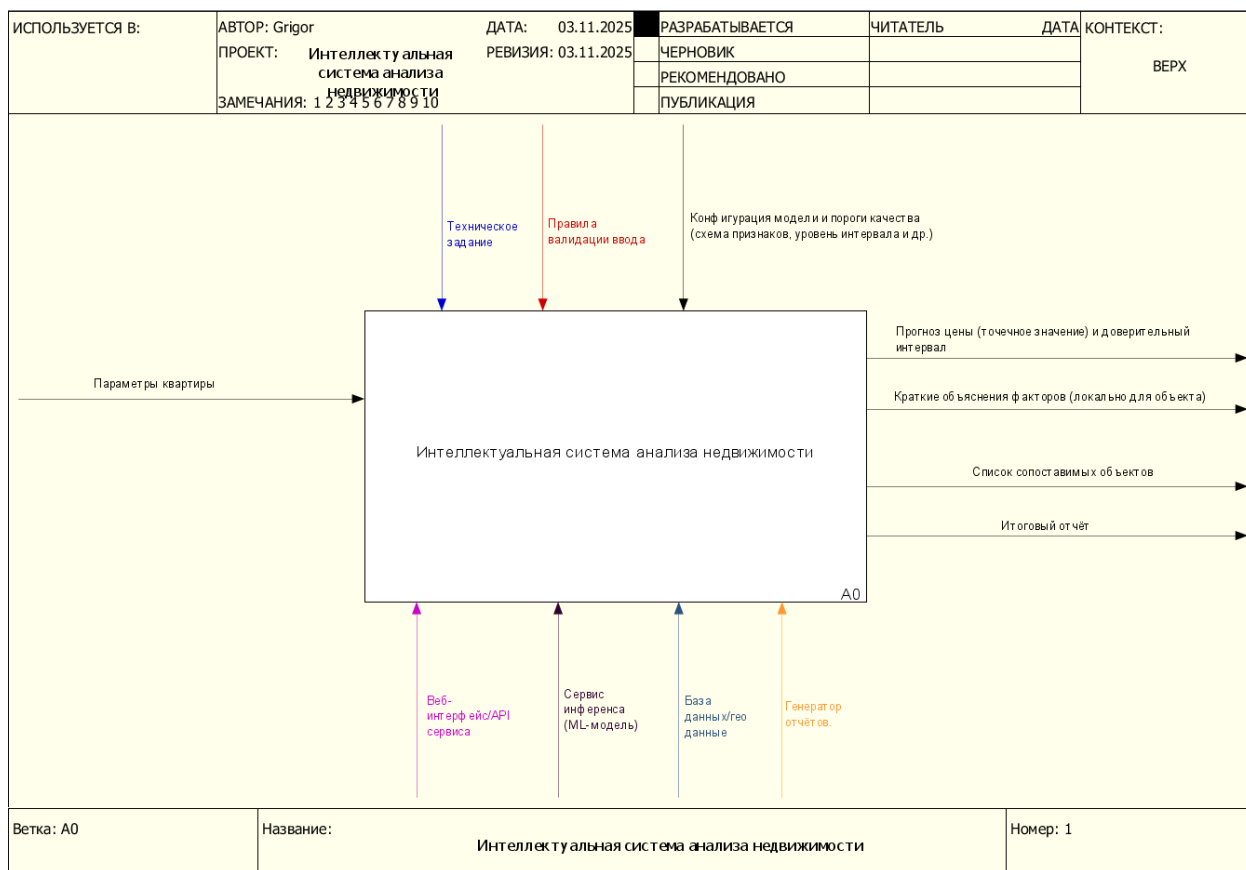


Рисунок 2.1 – Контекстный уровень «Интеллектуальная система анализа недвижимости»

Далее выполняется декомпозиция контекстного уровня на три ключевых процесса: подготовка и обогащение входа, прогноз и объяснение, формирование результата. Декомпозиция контекстного уровня представлена на Рисунке 2.2.

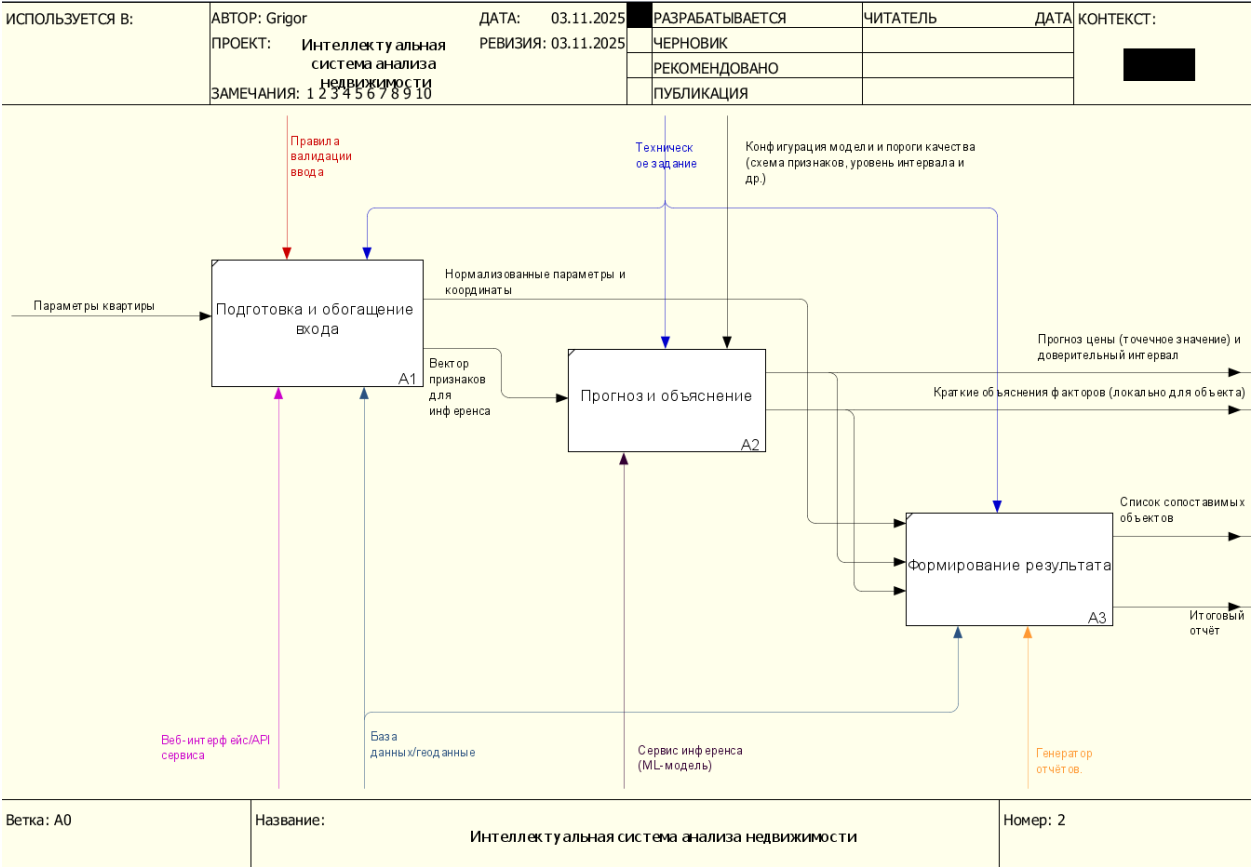


Рисунок 2.2 – Декомпозиция контекстного уровня

Процесс «Подготовка и обогащение входа». Выполняются приём параметров, геокодирование адреса и добавление простых геопризнаков (район/ячейка, близость транспорта). Результат – нормализованный объект с координатами и минимальным набором признаков.

Процесс «Прогноз и объяснение». По сформированному вектору признаков сервис инференса рассчитывает прогноз цены, строит доверительный интервал и формирует краткое локальное объяснение вклада основных факторов.

Процесс «Формирование результата». Подбираются 5-10 сопоставимых объектов, готовится карта и собирается итоговый отчёт (PDF/HTML) для пользователя.

Управляющее воздействие на функции A1-A3 оказывают ТЗ и правила валидации; реализация обеспечивается веб-интерфейсом, сервисом инференса, БД/геоданными и генератором отчётов. Поток данных организован

последовательно: из A1 в A2 передаётся нормализованное представление объекта, из A2 в A3 – результаты прогноза и параметры для подбора компараблов и сборки отчёта.

Далее приводится декомпозиция блока «Подготовка и обогащение входа», представленная на Рисунке 2.3.

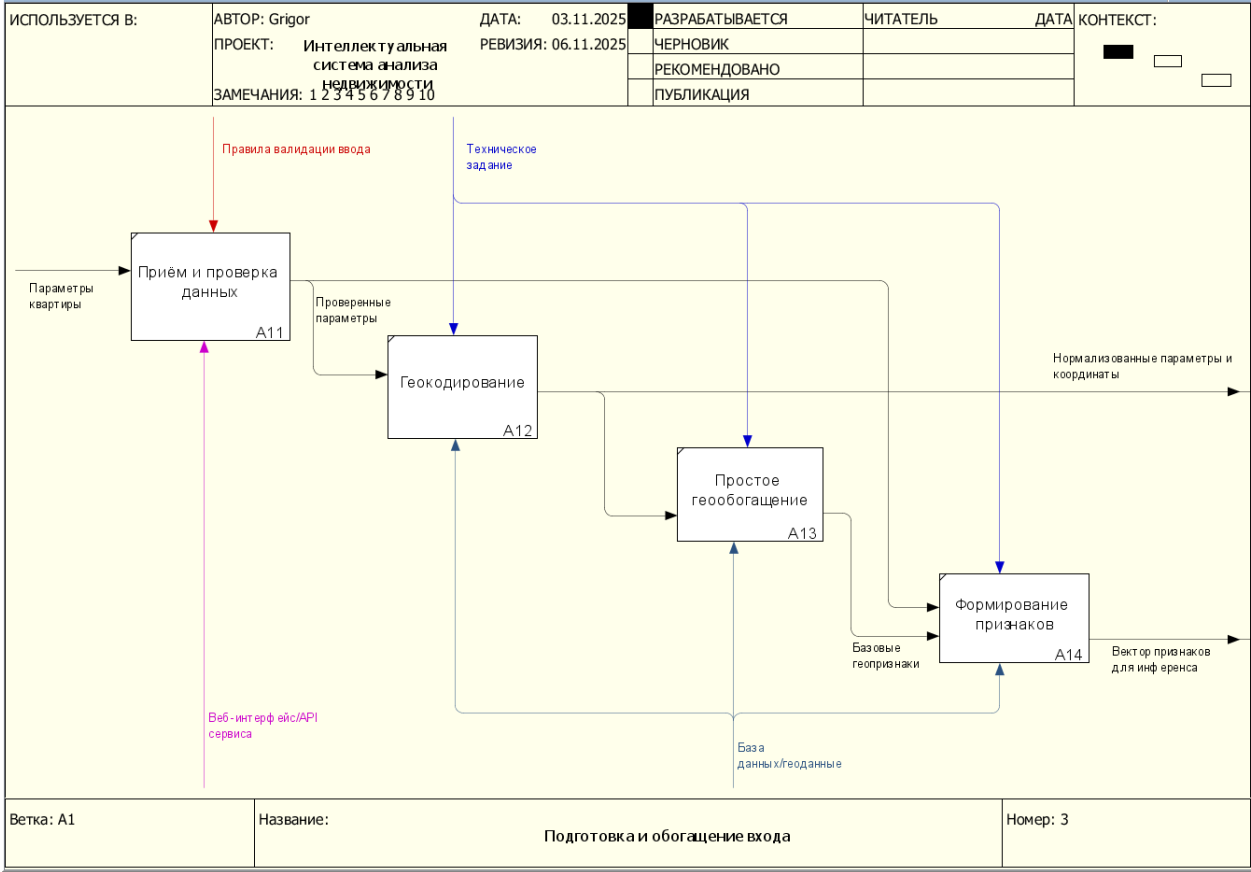


Рисунок 2.3 – Декомпозиция блока «Подготовка и обогащение входа»

Декомпозиция включает четыре подпроцесса:

- **A11 «Приём и проверка данных»**, получение параметров объекта через форму/API; базовая проверка обязательных полей и диапазонов. Выход: «Проверенные параметры»;
- **A12 «Геокодирование»**, преобразование адреса в координаты; проверка попадания в целевую территорию. Выход: «Нормализованные параметры и координаты»;

- **A13 «Простое геообогащение»**, определение района/ячейки локации и ближайшего транспорта; добавление базовых геопризнаков. Выход: «Базовые геопризнаки»;
- **A14 «Формирование признаков»**, объединение объектных и геопризнаков в минимальный вектор для модели; фиксация служебных меток. Выход: «Вектор признаков для инференса».

Для перечисленных подпроцессов управлениями выступают правила валидации и настройки геокодирования; механизмы – веб-слой, модуль геокодирования и БД/геоданные. Последовательность потоков следующая: принятый запрос из A11 поступает в A12, результат геокодирования направляет работу A13, полученное в A13 представление приводится в A14 и передаётся на этап A2 (прогноз и объяснение).

Таким образом, формируется простой и прослеживаемый конвейер: от ввода и геопривязки объекта – к расчёту прогноза с интервалом и краткими объяснениями – к подбору сопоставимых объектов и выпуску отчёта. Модель IDEF0 обеспечивает прозрачность этапов и соответствие требованиям, изложенным в техническом задании.

2.4 Анализ архитектуры системы

В качестве архитектуры интеллектуальной системы выбран трёхслойный модульный конвейер, включающий клиентский слой, сервис оценки и слой данных и артефактов. Внутри сервиса оценки используется оркестратор, который в зависимости от параметров запроса последовательно активирует необходимые модули: валидацию и геокодирование, генератор признаков, модуль инференса модели, интервальный модуль, модуль интерпретации и модуль подбора сопоставимых объектов. Такой подход позволяет гибко

настраивать последовательность обработки под конкретный сценарий, не изменяя ядро системы.

Обработка заявки начинается на клиентском слое. Веб-интерфейс или внешнее приложение по REST-API передаёт запрос в сервис оценки. На стороне сервиса оркестратор выполняет валидацию входных данных, при необходимости геокодирование адреса, формирует набор признаков на основе объектных характеристик и геоинформации, инициирует расчёт точечного прогноза арендной ставки, построение интервальной оценки и объяснение вклада признаков. Затем активируется модуль компараблов, который подбирает набор сопоставимых объектов по близости расположения и характеристик. Результаты объединяются и передаются в модуль визуализации и отчёта, формирующий ответ для пользователя (карта, ключевые показатели, краткий отчёт либо файл PDF/HTML). Архитектура интеллектуальной системы анализа недвижимости представлена на Рисунке 2.4.

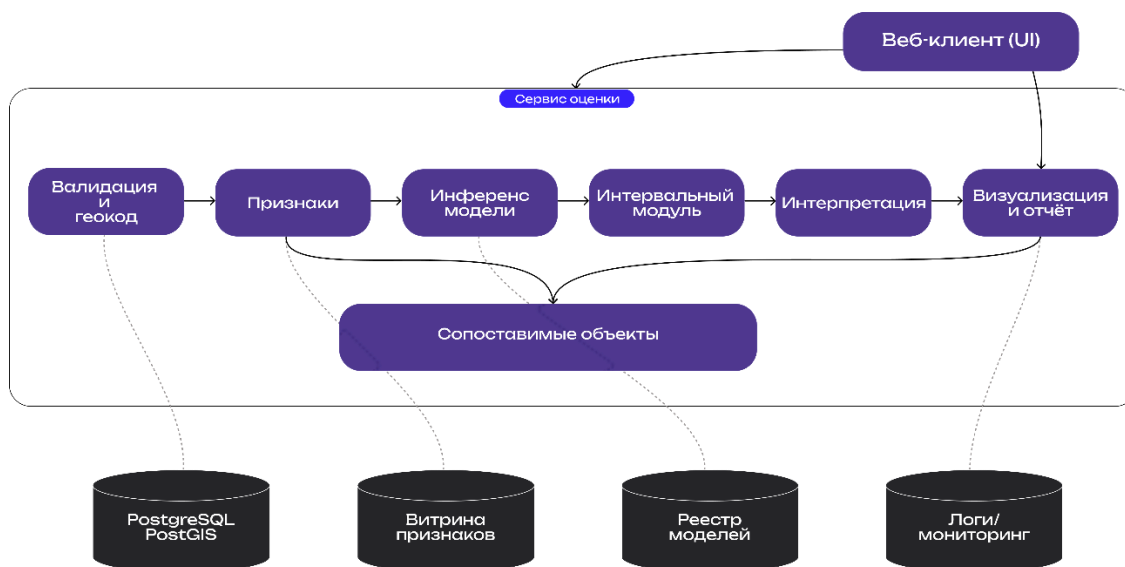


Рисунок 2.4 – Архитектура интеллектуальной системы анализа недвижимости

Выбор модульного конвейера и сервисной декомпозиции обусловлен следующими факторами:

- разнородность входа – пользователь может указать только адрес либо уже готовые координаты, часть атрибутов может отсутствовать;

конвейер запускает ровно те этапы, которые необходимы для данного запроса;

- поддержка интервальной оценки – использование квантильной и конформной схем формирования интервалов позволяет выбирать режим: более быстрые оценки либо интервалы с гарантированным покрытием при калибровке;
- производительность – ресурсоёмкие этапы (расширенное геообогащение, построение большого пула компараблов, вычисление объяснений) могут масштабироваться независимо и запускаться только при необходимости, что снижает среднее время отклика;
- расширяемость – новые геослои, признаки, модели и методы интерпретации подключаются как отдельные модули, что упрощает развитие системы без переработки оркестратора и клиентского слоя;
- трассируемость и аудит – версионирование данных и моделей, журналирование ветвей конвейера и конфигураций запуска обеспечивают воспроизводимость вычислений и соответствие требованиям технического задания;
- масштабируемость – инференс модели, подбор компараблов и формирование отчётов реализованы как стейтлесс-сервисы за API, что облегчает горизонтальное масштабирование под рост нагрузки.

Основные узлы архитектуры включают: веб-клиент и API-шлюз (приём и аутентификация запросов), оркестратор сервиса оценки, модуль валидации и геокодирования, генератор признаков (построение объектных и геопризнаков), модуль инференса модели (получение точечного прогноза), интервальный модуль (расчёт доверительного интервала), модуль интерпретации (формирование объяснимых оценок), модуль компараблов (фильтрация и ранжирование сопоставимых объектов), а также модуль визуализации и отчётов. На слое данных и артефактов располагаются хранилища PostgreSQL/PostGIS с пространственными слоями, витрина признаков, реестр моделей и подсистема

логирования и мониторинга, с которыми сервис оценки взаимодействует через стандартизированные интерфейсы.

2.5 Вывод по аналитическому разделу

В аналитическом разделе последовательно уточнены ключевые аспекты построения интеллектуальной системы анализа недвижимости. Сформулированы цели и ограничения решения с позиции технического задания и практических рисков данных: неоднородность входной информации, пространственно-временная зависимость цен, необходимость прозрачных объяснений и корректной оценки неопределённости. Зафиксированы базовые критерии качества: воспроизводимость результатов (версионирование данных и моделей), честность оценки (валидные сплиты по времени и пространству), прикладные метрики прогноза (MAE, RMSE, MAPE на лог-шкале), метрики интервалов (покрытие и средняя ширина), а также эксплуатационные характеристики (время отклика, возможность горизонтального масштабирования).

Обоснован выбор методов: градиентные ансамбли для табличных данных с расширенным геопризнаковым описанием, интервал прогноза через квантильный и/или конформный подход, интерпретация результатов с помощью SHAP, модуль подбора сопоставимых объектов по сочетанию геоблизости и сходства атрибутов. Описана методология контроля качества: временное разбиение, пространственная блочная валидация, мониторинг дрейфа и обработка «вне распределения». Закреплён практический стек реализации (управление экспериментами, оптимизация гиперпараметров, витрина признаков и пространственные слои, лёгкий сервис инференса и генерация отчётов), обеспечивающий быстрый цикл «данные → модель → сервис».

Архитектура решения представлена как модульный конвейер с опциональными ветвями (режим интервала, профиль сделки, уровень

геообогащения, канал выдачи). Функциональная логика процесса формализована в нотации IDEF0: от подготовки и обогащения входа – к прогнозу и объяснениям – к формированию результата (компараблы, карта, отчёт). Такой подход обеспечивает прослеживаемость потоков, изоляцию компонентов, расширяемость и соответствие требованиям ТЗ.

В результате сформирована целостная концепция системы – от выбора методов и инструментов до схемы обработки и взаимодействия компонентов, – готовая к реализации и экспериментальной проверке в технологическом разделе по метрикам качества прогноза и интервалов, производительности и устойчивости к изменению данных.

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В данном разделе представлено детальное описание реализации интеллектуальной системы анализа недвижимости. Рассматриваются программные сущности и их роли в реализации архитектуры: сервис приёма запросов, модуль валидации и геокодирования, генератор признаков, модуль инференса (прогноз + интервальная оценка), подсистема интерпретации (локальные объяснения), модуль подбора сопоставимых объектов, а также компоненты визуализации и формирования отчёта. Показано, как перечисленные модули реализуют описанные бизнес-процессы (IDEF0), какие данные они принимают на вход, какие результаты формируют и как взаимодействуют через API и хранилища.

Отдельное внимание уделено качеству и надёжности: приводятся сценарии и результаты тестирования (модульного, интеграционного, системного и нагрузочного, а также ручной приёмки), описывается методика расчёта надёжности программного продукта после финальной сборки. Представлено руководство пользователя с типовыми вариантами использования (use-cases), формами ввода и сообщениями об ошибках/предупреждениях. В завершение раздела приведены фактические показатели работы системы: точность и покрытие интервальных предсказаний, время отклика, требования к ресурсам и объёмам хранилищ, что позволяет оценить соответствие решения требованиям ТЗ и его прикладную ценность.

3.1 Подготовка данных для обучения модели

Для обучения моделей был сформирован собственный датасет на основе публичных объявлений о аренде жилья и открытых геоданных (улично-дорожная сеть, объекты инфраструктуры). Сбор осуществлялся в учебных целях

с соблюдением ограничений источников (частота запросов, указание источника). Сырые данные сохранялись в базу данных.

Был разработан скрипт агрегации и конвертации выгрузок/страниц в унифицированную схему: `listing_id`, тип сделки (аренда), валюта и цена, адрес/координаты (если есть), площадь(и), комнаты, этаж/этажность, тип/материал дома, состояние/ремонт, дата публикации/обновления, текст описания. На этапе нормализации выполнялось:

- приведение цен к рублям и расчёт «ставки в мес.» для аренды;
- унификация категорий (тип дома, материал, состояние) по словарям соответствия;
- парсинг площадей (общая/жилая/кухня), аккуратная работа с единицами измерения.

Адреса преобразовывались в координаты с кэшированием результатов. На основе координат рассчитывались базовые геопризнаки:

- индекс НЗ заданного разрешения [28] и принадлежность к административному району;
- ближайшие узлы транспорта (метро/МЦД) и оценка доступности (приблизённое время в пути/расстояние);
- показатели окружения по ячейке/району: медианы цен, плотность объявлений, доля квартир разных классов/комнатности, «темпы» изменения цены за недавний период.

Для хранения пространственных слоёв использовался PostGIS; агрегаты по НЗ формировались партиями и версионировались.

Для аренды целевая переменная – месячная ставка. Для стабилизации дисперсии дополнительно готовилась лог-шкала целевых.

Признаки включали:

- объектные (площади, комнаты, этаж/этажность, тип/материал/год, состояние);
- геопризнаки (НЗ, расстояния/доступность, агрегаты рынка);

- текстовые маркеры из описаний (простые фичи: «ремонт», «мебель», «вид», «балкон» и пр.).

Чтобы избежать утечки и завышения метрик, применялось:

- временное разбиение: обучение на прошлых периодах, тест – на более поздних публикациях;
- пространственная блок-валидация: блоки по НЗ/районам для честной оценки переносимости между локациями.

Фиксировались состав фичей, пороги фильтрации и даты сплитов – для воспроизводимости экспериментов.

Перед экспортом в витрину выполнялось профилирование: доли пропусков, распределения ключевых полей, частота дублей после дедупликации, разметка редких категорий. Для «проблемных» записей формировались отчётные выборки на ручную проверку.

Итогом подготовки являлись артефакты данных:

- features/train.parquet, splits/valid.parquet, splits/test.parquet – витрина признаков;
- listings_rent.parquet – целевые для аренды;
- служебные таблицы агрегатов по НЗ/районам и журнал преобразований (дата, версия витрины, правила очистки).

Такой конвейер «сбор → очистка → геокодирование → геообогащение → витрина → сплиты» обеспечивает воспроизводимость обучения, сопоставимость экспериментов и соответствие требованиям ТЗ к данным и процессу их подготовки.

3.2 Обучение нейронной сети

Для решения задачи прогноза **месячной ставки аренды** были обучены две линии моделей:

1. Базовая – градиентный бустинг над деревьям решений (LightGBM/CatBoost) как эталон для сравнения качества и объяснимости;
2. Основная – табличная нейронная сеть (MLP), настроенная под витрину признаков и поддерживающая формирование доверительных интервалов.

Ниже описаны архитектура, подготовка признаков к обучению, схема разбиений данных, режимы формирования интервалов, метрики, подбор гиперпараметров и сохраняемые артефакты.

Градиентный бустинг (baseline) использовался как точка отсчёта благодаря устойчивости на табличных данных с числовыми/категориальными/геопространственными признаками, высокой скорости инференса и нативной поддержке квантильной регрессии. Для CatBoost – встроенное кодирование категорий и ordered boosting, для LightGBM – гистограммный алгоритм и быстрый подбор глубины/листов. Бустинг применялся как:

- модель точечного прогноза медианы (по лог-таргету);
- модель квантильной регрессии (нижний/верхний квантили $\tau=0.1/0.9$) для формирования интервалов.

Нейронная сеть (MLP) – лёгкая полносвязная архитектура для табличных данных: вход – конкатенация нормализованных числовых признаков и закодированных категориальных; сети – 2–4 полносвязных слоя с BatchNorm и Dropout; активации – ReLU/SiLU. Выходной блок реализован в двух вариантах:

- многоцелевой «квантильный» выход: три головы для $\tau=0.1 / 0.5 / 0.9$ (нижний, медианный, верхний прогноз в лог-шкале);
- точечный выход + конформная калибровка: одна голова даёт медиану, а интервалы формируются на отдельной калибровочной выборке.

Выбор между вариантами фиксировался по валидации: при приоритете стабильного покрытия – конформная калибровка; при приоритете скорости – квантильные головы.

Режим формирования доверительных интервалов:

1. Квантильная регрессия; в бустинге – отдельные модели/режимы для нижнего и верхнего квантиля; в MLP – многоцелевой выход с quantile loss; интервал формируется как [нижний, верхний] в лог-шкале с последующим обратным преобразованием;
2. Конформная калибровка (split conformal); на калибровочной части измеряется распределение остатков в лог-шкале; по заданному уровню покрытия (90%) вычисляется поправка ширины интервала; применимо как к бустингу (точечная модель), так и к MLP.

Функции потерь:

1. Точечный прогноз: MSE по лог-таргету.
2. Квантильные головы: quantile loss для $\tau=0.1, 0.5, 0.9$.

Точечные метрики:

1. MAE (руб.) – средняя абсолютная ошибка после обратного лог-преобразования (текущая модель LightGBM: 14 371 руб. на валидации).
2. MAPE (%) – средняя абсолютная процентная ошибка.
3. RMSE (лог-шкала) – корень из среднеквадратичной ошибки по $\log(\text{rent})$ (текущая модель: 24 303 руб. на валидации).
4. R^2 (лог-шкала) – опционально, для оценки доли объяснённой дисперсии.

Значения метрик фиксируются для валидации и финального теста.

Обучение и подбор гиперпараметров:

1. MLP: оптимизатор Adam/AdamW; ранняя остановка по валидации; Dropout и L2-регуляризация; scheduler шага обучения.

2. Бустинг: `n_estimators`, глубина/`num_leaves`, `learning_rate`, минимальные выборки в листе, L2/категориальные параметры – подбирались.
3. Подбор: Optuna/поисковая сетка с целью минимизации MAE по лог-таргету на валидации; фиксирование `random_state` и ограничений на сложность для устойчивости.
4. Воспроизводимость: журнал MLflow (параметры, метрики, версии данных/фичей, артефакты), фиксированные `seeds`, сериализация препроцессоров.

Интерпретируемость:

1. Для бустинга – TreeSHAP (быстрая и точная локальная/глобальная интерпретация).
2. Для MLP – KernelSHAP на подвыборке признаков (общий метод для моделей без «деревянной» структуры).
3. Локальные объяснения показываются пользователю вместе с прогнозом и интервалом; агрегированные важности – в административном отчёте качества.

В реестр моделей сохраняются: веса/конфиг модели (MLP или бустинг), нормировщики/кодировщики, параметры интервалов (квантильные уровни или конформная поправка), отчёт о метриках, версия витрины и сплитов.

Финальный выбор модели инференса (бустинг или MLP) выполняется по совокупности качества, стабильности покрытия и времени отклика на отложенном тесте; выбранная конфигурация помечается как `production` и используется сервисом предсказаний.

3.3 Описание программной реализации

Ниже приведено описание ключевых программных сущностей, отвечающих за реализацию модулей системы: ML-сервис (`realval`), `backend-`

шлюз (go_back) и веб-интерфейс (UI). Описано, какие компоненты они реализуют, какие процессы выполняют и какие данные хранят/возвращают.

3.3.1 ML-сервис

Автономный сервис инференса и вспомогательных операций (обучение, калибровка интервала, формирование отчётов). Работает поверх подготовленной витрины признаков и артефактов модели.

Общая структура указана на Рисунке 3.1:

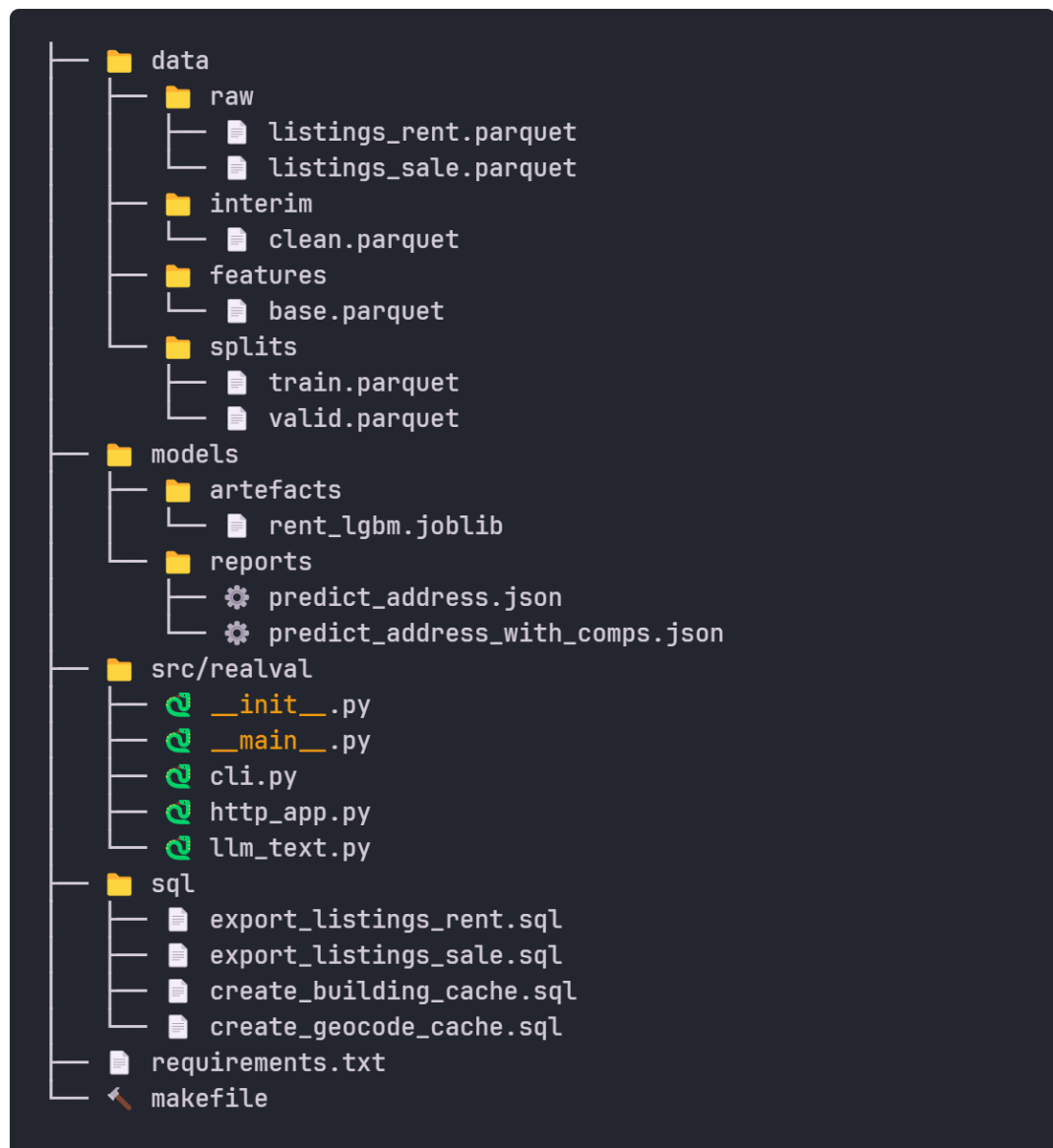


Рисунок 3.1 – Общая структура ML-сервиса

Основные сущности:

1. «src/realval/http_app.py» – HTTP-приложение инференса, принимает запрос оценки аренды (адрес/координаты и основные параметры квартиры), запускает последовательность: валидация → построение вектора признаков → прогноз медианы → расчёт доверительного интервала (квантильный/конформный режим) → формирование кратких локальных объяснений → (опционально) подбор сопоставимых объектов → сборка ответа в JSON;
2. «src/realval/cli.py, src/realval/__main__.py» – CLI-обёртка, команды для пакетных задач (обучение/переобучение, валидация, экспорт артефактов, локальный инференс для набора адресов, генерация отчётов качества);
3. «src/realval/llm_text.py» – генерация пояснений (короткое текстовое резюме факторов на основе рассчитанных вкладов и описательных полей объекта), собирает из числовых объяснений краткий «комментарий к оценке», пригодный для отчёта;
4. «models/artefacts/rent_lgbm.joblib» – артефакт базовой модели (градиентный бустинг для аренды);
5. «models/reports/*.json» – примеры ответов (прогноз по адресу, с компараблами и без);
6. «data/» – витрина и сплиты: «data/features/base.parquet, data/splits/{train,valid}.parquet», «data/raw/listings_rent.parquet» – согласованные наборы, используемые при инференсе/валидации;
7. «sql/*.sql» – служебные запросы для экспорта/кеша геоданных и объявлений (используются на этапе подготовки/обслуживания витрины);

3.3.2 Backend-шлюз

Единая точка входа для фронтенда и внешних интеграций; инкапсулирует валидацию/геокодирование, обращение к ML-сервису и формирование PDF-отчёта. Сервис stateless; состояние (история запросов, кэши геокодирования) хранится в базе.

Общая структура указана на Рисунке 3.2:

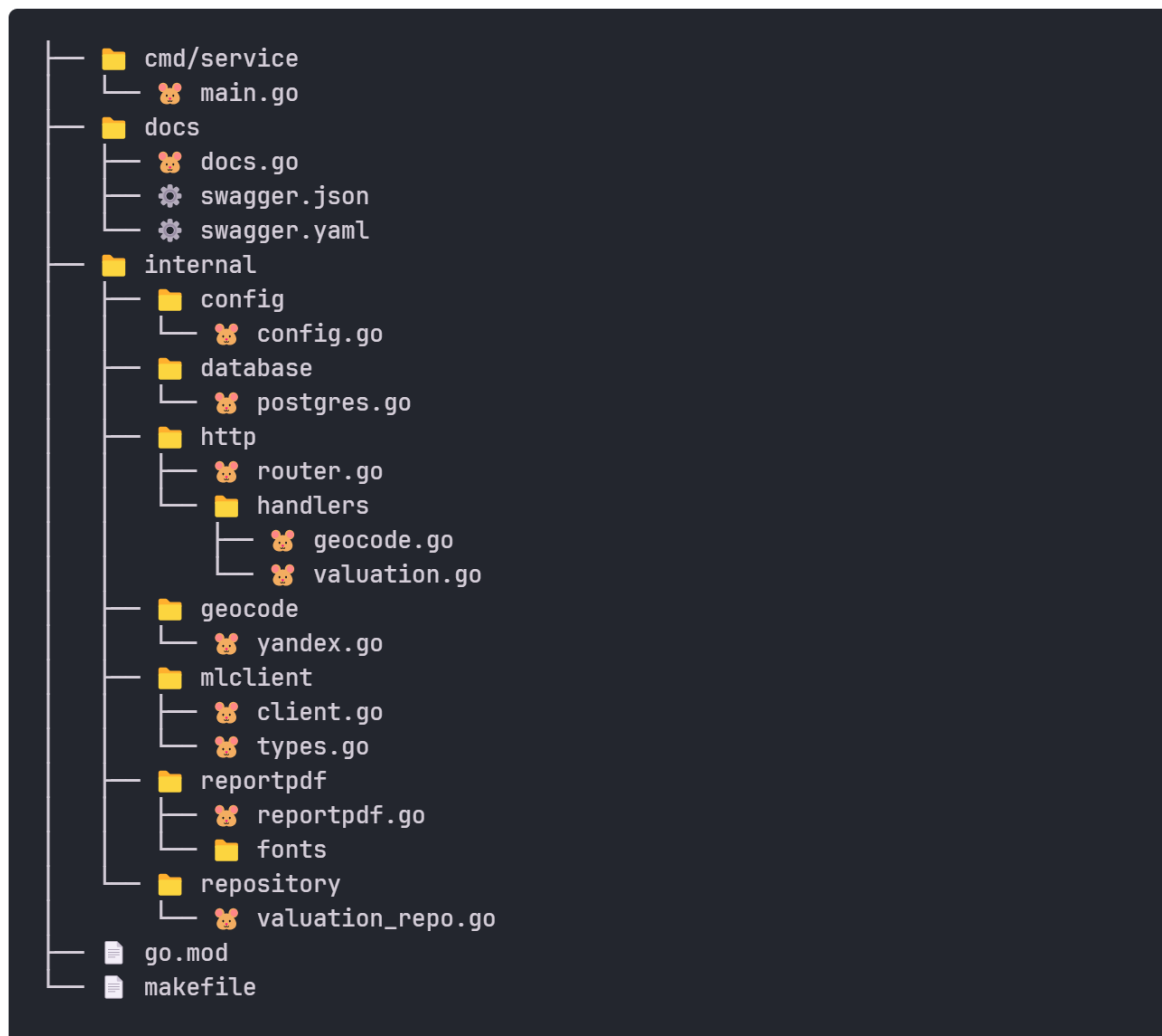


Рисунок 3.2 – Общая структура Backend-шлюза

Основные сущности:

1. «`cmd/service/main.go`» – входная точка сервиса, которая инициализирует конфигурацию, логгер, подключение к БД, маршрутизатора HTTP;

2. «internal/http/router.go» – маршрутизатор REST, который регистрирует эндпоинты уровня «/geocode», «/valuation»;
3. «internal/http/handlers/geocode.go» – обработчик геокодирования, принимает текст адреса, вызывает провайдера геокодера, возвращает координаты и нормализованный адрес;
4. Провайдеры «internal/geocode/yandex.go»; кэш в БД – подсказка для ввода адреса.
5. «internal/http/handlers/valuation.go» – обработчик оценки аренды, Принимает параметры объекта (адрес/координаты, площади, комнаты, этаж/этажность, тип/состояние и т. п.), валидирует вход, вызывает ML-клиент для расчёта прогноза и интервала, при необходимости – с подбором компараблов; возвращает структурированный JSON и/или запускает генерацию PDF;
6. «internal/mlclient/{client.go, types.go}» – клиент для realval, который инкапсулирует HTTP-вызовы к ML-сервису (эндпоинты предсказания, объяснений, компараблов), преобразование типов и контроль таймаутов/повторов;
7. «internal/reportpdf/reportpdf.go» (+ fonts/) – генератор PDF-отчёта, который собирает отчёт (цена/интервал, топ-факторы, карта/список компараблов) с применением фирменного шрифта и полей; отдаёт файл на скачивание/сохранение;
8. «internal/repository/valuation_repo.go» – репозиторий, это доступ к БД: сохранение запросов/ответов оценки (без ПДн), кэшей геокодирования, журналов служебных операций;
9. «internal/database/postgres.go» – инициализация подключения к БД;
10. «internal/config/config.go» – загрузка конфигурации (порты, DSN, адреса сервисов);
11. «internal/logger/logger.go» – структурный логгер и уровни логирования;

12. «docs/swagger.*», «docs/docs.go» – описание REST-контрактов (автогенерация Swagger/OpenAPI);

3.3.3 Веб-интерфейс

Веб-интерфейс реализован как одностраничное приложение на React и Next.js с использованием Tailwind CSS [29] [30] [31]. Одностраничный интерфейс для ввода параметров квартиры, вызова оценки и просмотра результата с компараблами и скачиванием отчёта.

Общая структура указана на Рисунке 3.3

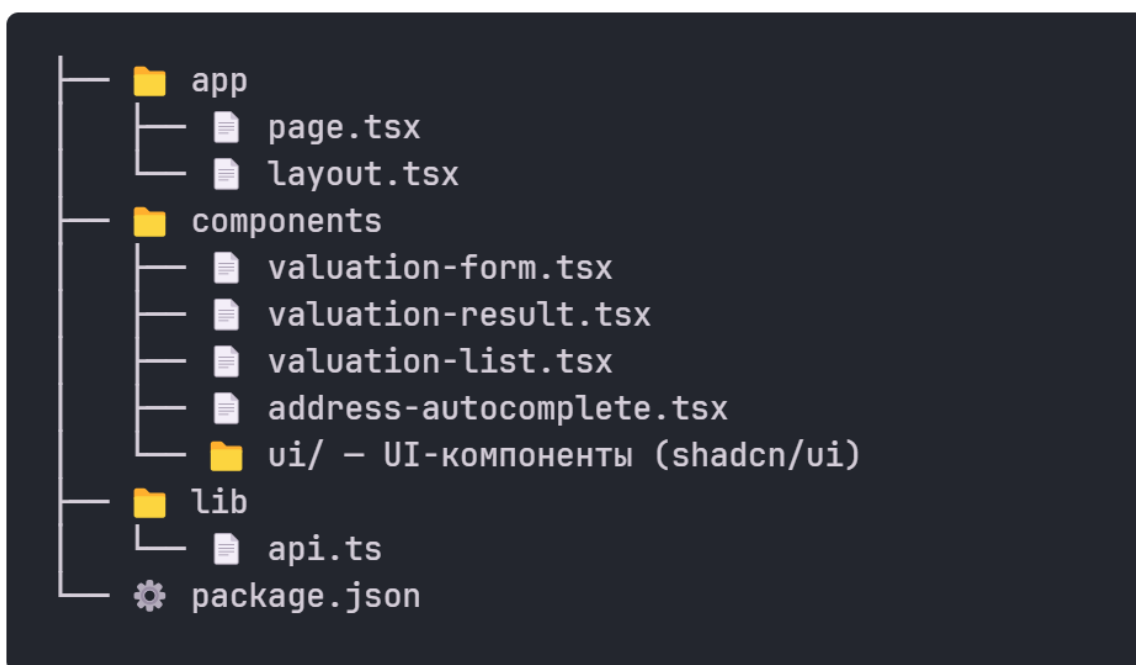


Рисунок 3.3 – Общая структура Веб-интерфейса

Основные сущности:

1. «app/page.tsx, app/layout.tsx» – каркас страницы приложения.
2. «components/valuation-form.tsx» – форма оценки (адрес/координаты, параметры объекта, выбор режима интервала).
3. «components/address-autocomplete.tsx» – подсказки адресов (вызовы «/geocode»).
4. «components/valuation-result.tsx» – отображение результата (цена/интервал, факторы, список компараблов, карта).

5. «components/valuation-list.tsx» – история/повторный вызов (если включено).
6. «lib/api.ts» – клиент REST к backend; lib/utls.ts – утилиты форматирования значений.
7. «public/», «globals.css», конфигурации (package.json, tailwind.config.ts, next.config.js) – статические ресурсы и стили.

После успешного запуска пользователю доступен главный экран веб-интерфейса, представленный на Рисунке 3.4. В верхней части расположены логотип и индикатор состояния ML-сервиса («ML сервис: В сети»), что позволяет оперативно оценить готовность подсистемы инференса. Центральный блок содержит заголовок и краткое описание назначения системы. Ниже размещены карточки с ключевыми преимуществами: точный прогноз, диапазон уверенности (доверительный интервал) и сопоставимые объекты. Под карточками — панель действий с кнопками «Новая оценка» и «Общий список» (история обращений, при включении соответствующей опции).

RentAdvisor
Нейросетевой прогноз стоимости аренды

ML сервис: В сети

Оценка рыночной стоимости аренды квартир в Москве

RentAdvisor использует современные алгоритмы машинного обучения для точного прогнозирования стоимости аренды недвижимости. Система анализирует множество факторов: расположение, площадь, состояние квартиры, близость к метро и другие параметры для расчета оптимальной рыночной ставки.

Точный прогноз
Нейронная сеть обучена на тысячах объявлений о аренде в Москве

Диапазон уверенности
Получите не только точечный прогноз, но и доверительный интервал цены

Сопоставимые объекты
Сравните прогноз с реальными предложениями на рынке

Дипломный проект РТУ МИРЭА | Кафедра Вычислительной техники

☑ Новая оценка ☰ Общий список

Оценка стоимости аренды

Заполните параметры квартиры для получения прогноза стоимости аренды

Адрес *	Город *
Нежинская улица, 1к1	Москва
Количество комнат * (0 - студия)	Общая площадь (м²) *
2	90
Жилая площадь (м²)	Площадь кухни (м²)
65	20
Этаж	Этажей в доме
...	...

Рисунок 3.4 – Главный экран веб-интерфейса системы оценки аренды

Ниже расположен основной раздел «Оценка стоимости аренды» с формой ввода параметров объекта (Рисунок 3.5). Форма поддерживает автодополнение адреса, валидацию обязательных полей и удобное разделение атрибутов по

колонкам. Пользователь указывает: адрес и город, количество комнат, общую/жилую/кухонную площадь, этаж и этажность, год постройки, материал дома, а также состояние. Дополнительно доступен переключатель «Текстовый анализ с AI» для генерации развёрнутого пояснения. Запрос запускается кнопкой «Рассчитать стоимость»; по результату система формирует точечный прогноз месячной ставки, доверительный интервал и краткие локальные объяснения факторов, а также (по запросу) список сопоставимых объектов и PDF-отчёт.

Дипломный проект РТУ МИРЭА | Кафедра Вычислительной техники

🔍 Новая оценка ≡ Общий список

Оценка стоимости аренды

Заполните параметры квартиры для получения прогноза стоимости аренды

Адрес *	Город *
Нежинская улица, 1к1	Москва
Количество комнат * (0 - студия)	Общая площадь (м²) *
2	90
Жилая площадь (м²)	Площадь кухни (м²)
65	20
Этаж	Этажей в доме
24	31
Год постройки	Материал дома
2008	Монолит
Состояние	
Выберите состояние	
Текстовый анализ с AI Добавить развернутое описание и факторы оценки	
☒	
Рассчитать стоимость	

Рисунок 3.5 – Форма ввода параметров объекта для расчёта аренды

Интерфейс выполнен в едином тёмном оформлении, адаптирован под разные разрешения экрана, а статусные элементы (индикатор ML-сервиса, подсказки/ошибки валидации, состояния загрузки) обеспечивают прозрачную обратную связь при работе пользователя.

3.4 Тестирование приложения

В ходе тестирования проверялись функциональные, интеграционные и системные аспекты работы системы, а также корректность расчёта оценок и производительность сервиса инференса. Тестовые сценарии покрывают основные пользовательские потоки (геокодирование, расчёт оценки аренды,

интервалы неопределённости, подбор компараблов, генерация PDF-отчёта), обработку ошибок валидации и граничные случаи. Для удобства навигации в Таблице 3.1 приведён перечень тест-кейсов (ID и наименование), а в последующих таблицах описаны шаги выполнения, тестовые данные и ожидаемые результаты.

Таблица 3.1 – Перечень тест-кейсов

ID	Название
T-01	Геокодирование валидного адреса
T-02	Обработка некорректного адреса
T-03	Валидация обязательных полей
T-04	Точечный прогноз и интервал
T-05	Подбор компараблов (3–5 шт.)
T-06	Генерация PDF-отчёта
T-07	Получение списка оценок
T-08	Получение оценки по ID
T-09	Проверка health ML
T-10	Интеграция UI → Backend

Далее приведены подробные тестовые сценарии для указанных кейсов. В Таблице 3.2 рассматриваются проверки корректности геокодирования и базовой валидации входных данных.

Таблица 3.2 – Геокодирование и валидация ввода

ID	Локализация	Шаги	Тестовые данные	Ожидаемый результат	Статус
T-01	`GET /api/v1/geocode/suggest`	Ввести адрес и отправить запрос	`query=Москва, Тверская 7`	Координаты + нормализованный адрес, HTTP 200	✓
T-02	`GET /api/v1/geocode/suggest`	Ввести мусорную строку	`query=zzzz`	Возврат результатов (похожих мест) или пустой массив	✓
T-03	`POST /api/v1/valuation/addresses`	Отправить запрос без площади	адрес + комнаты, без `area_total`	HTTP 400, ошибка валидации поля	✓

Результаты Таблицы 3.2 показывают, что сервис геокодирования корректно обрабатывает как корректные, так и некорректные запросы, а также возвращает ошибку валидации при отсутствии обязательных параметров. В

Таблице 3.3 приведены тесты основного функционала — расчёта оценки аренды, формирования доверительного интервала и подбора сопоставимых объектов.

Таблица 3.3 – Расчёт оценки, интервалы неопределённости и компараблы

ID	Локализация	Шаги	Тестовые данные	Ожидаемый результат	Статус
T-04	`POST /api/v1/valuation/address`	Запрос оценки с полными данными	`{"city":"Москва", "address":"Тверская улица, 7","rooms":2,"area_total":60}`	JSON с `report_id`, `price.prediction_rub`, `price.interval_low_rub` /`high_rub`, HTTP 200	✓
T-05	`POST /api/v1/valuation/address`	Запрос оценки	валидный объект	3-5 объектов в `comparables[]`: price, rooms, area, distance_km, metro, url	✓
T-06	`GET /api/v1/valuation/{id}/pdf`	Скачать PDF по report_id	`id` из T-04	PDF с ценой, интервалом, компараблами; время ≤5 сек	✓

По результатам Таблицы 3.3 подтверждена корректность основного расчётного контура: формируется точечный прогноз, интервальная оценка и список компараблов, а также обеспечивается генерация PDF-отчёта в заданное время. В Таблице 3.4 рассмотрены сценарии работы с сохранёнными результатами оценок и получение отчётов по идентификатору.

Таблица 3.4 – Генерация отчётов и работа с сохранёнными оценками

ID	Локализация	Шаги	Тестовые данные	Ожидаемый результат	Статус
T-07	`GET /api/v1/valuation`	Запросить список всех оценок	—	HTTP 200, массив объектов с `id`, `city`, `address`, `created_at`, `prediction_rub`	✓
T-08	`GET /api/v1/valuation/{id}`	Запросить отчет по ID	`id=54f7cda8-...`	HTTP 200, полные данные оценки с объектом, ценой, компараблами	✓

Результаты Таблицы 3.4 подтверждают корректность доступа к истории расчётов и получения полной информации по конкретному отчёту. В Таблице 3.5

приведены интеграционные и системные проверки, подтверждающие работоспособность сервисов в связке и корректность работы пользовательского интерфейса.

Таблица 3.5 – Интеграционное и системное тестирование

ID	Локализация	Шаги	Ожидаемый результат	Статус
T-09	`GET /api/v1/valuation/ml-health`	Проверка доступности ML	HTTP 200, <code>{"status":"ok"}</code>	✓
T-10	UI форма оценки	Заполнить форму и отправить	Отображение результата с ценой, интервалом, компараблами	✓

Интеграционные проверки показали, что компоненты системы корректно взаимодействуют между собой: ML-сервис доступен, а пользовательский интерфейс успешно инициирует расчёт и отображает результат. Таким образом, проведённые тесты подтверждают работоспособность основных функций системы и соответствие сценариям использования, описанным в руководстве пользователя.

3.5 Расчёт надёжности

Для количественной оценки качества прогноза стоимости аренды были рассчитаны стандартные метрики для задач регрессии: средняя абсолютная ошибка (MAE), корень из средней квадратичной ошибки (RMSE) и средняя абсолютная процентная ошибка (MAPE) [32]. Метрики вычислялись на отложенной тестовой выборке, не участвующей в обучении и подборе гиперпараметров.

Средняя абсолютная ошибка определяется как

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (3.1)$$

где y_i – фактическая ставка аренды для объекта i ,

$\hat{y}_i$ – предсказанное значение,

n – количество объектов в выборке.

В проведённых экспериментах получено значение $MAE \approx 14371,29$ руб., то есть в среднем модель ошибается на ~14 тыс. руб. в месяц. Для типичного уровня арендной платы это соответствует умеренному отклонению и позволяет использовать модель как ориентир при принятии решений.

Корень из средней квадратичной ошибки рассчитывается по формуле

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (3.2)$$

Полученное значение $RMSE \approx 24303,22$ руб. показывает, что крупные ошибки (сильно переоценённые или недооценённые объекты) встречаются реже, но дают больший вклад в квадратичную метрику. $RMSE$, как более «строгая» метрика, используется для контроля хвостов распределения ошибок и подбора конфигурации модели.

Средняя абсолютная процентная ошибка определяется как

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|, \quad (3.3)$$

В эксперименте получено $MAPE \approx 14,86\%$, то есть относительная ошибка прогноза в среднем находится на уровне ~15%. В рамках требований

технического задания условие «точность не менее 60%» интерпретируется как средняя относительная ошибка не хуже 40%. Фактическое значение MAPE существенно лучше этого порога, что говорит о запасе по качеству модели и её пригодности для практического использования.

Дополнительно для интервальных предсказаний оцениваются показатели покрытия (доля фактических значений, попавших в доверительный интервал) и средней ширины интервала; эти метрики использовались при настройке параметров квантильной и конформной моделей. В совокупности полученные значения показателей качества подтверждают, что модель удовлетворяет требованиям ТЗ по точности и может служить надёжной основой для работы сервиса оценки стоимости аренды.

3.6 Руководство пользователя

Данный раздел описывает работу пользователя с веб-сервисом RentAdvisor (оценка рыночной ставки аренды квартир). Руководство ориентировано на конечных пользователей и базируется на реальных сценариях использования.

3.6.1 Доступ и требования:

1. Современный браузер: Chrome/Edge/Firefox (актуальные версии), включён JavaScript.
2. Подключение к интернету, установка дополнительного ПО не требуется.
3. Адрес интерфейса – главная страница сервиса (см. Рисунок 3.4) и форма ввода параметров (см. Рисунок 3.5).

3.6.2 Основные варианты использования

В данном подразделе приведены типовые сценарии работы пользователя с сервисом. Каждый вариант использования описывает цель, последовательность действий и ожидаемый результат, что позволяет быстро понять, как получить оценку стоимости аренды и проверить доступность ML-сервиса.

3.6.2.1 Расчёт стоимости по адресу

Цель. Получить прогноз, доверительный интервал и список сопоставимых объектов.

Порядок действий:

1. Заполните параметры объекта (этаж/этажность, жилую/кухонную площадь, год постройки, материал, состояние).
2. Включите опцию «Текстовый анализ с AI» (по желанию — для развёрнутого комментария к оценке).
3. Нажмите «Рассчитать стоимость».

Результат: кроме прогноза и интервала, отобразятся 5–10 компараблов с расстоянием/временем до объекта и ключевыми атрибутами; доступна кнопка «Скачать отчёт (PDF)».

3.6.2.2 Проверка статуса сервиса

Цель: понять, готов ли ML-сервис к работе.

Порядок действий:

1. Смотрите индикатор в шапке: «ML сервис: В сети» / «Недоступен».
2. При недоступности — повторите попытку позже или обратитесь к администратору.

3.6.3 Порядок работы

1. Откройте главный экран. Убедитесь, что ML-сервис «В сети».
2. Перейдите к разделу «Оценка стоимости аренды».
3. Заполните адрес и обязательные параметры (город, комнаты, общая площадь).
4. При необходимости заполните расширенные поля и включите опцию AI-комментарий.
5. Нажмите «Рассчитать стоимость» и дождитесь результата.
6. Просмотрите прогноз, доверительный интервал и объяснения. При необходимости скачайте PDF-отчёт.

3.6.4 Рекомендации по корректному вводу данных

1. Адрес указывайте в формате город, улица, дом (при наличии — корпус/строение).
2. Проверяйте согласованность этаж/этажность и реалистичность площадей.
3. Если описание квартиры содержит особенности (ремонт, меблировка), укажите состояние и факторы в доступных полях формы.

3.6.5 Получение отчёта

После успешного расчёта становится доступной кнопка «Скачать отчёт (PDF)».

Отчёт содержит:

- точечный прогноз стоимости аренды;
- доверительный интервал;
- краткие объяснения факторов, влияющих на оценку;

- список сопоставимых объектов.

3.6.6 Политика данных

Сервис не запрашивает персональные данные пользователей. Адрес и параметры объекта используются только для расчёта оценки; при включённой истории запросов сохраняются обезличенные записи (без ПДн) для улучшения качества и аудита.

3.7 Вывод по технологическому разделу

В технологическом разделе реализована и описана прикладная система оценки ставки аренды: подготовка данных и витрины, обучение базовой (градиентный бустинг) и нейросетевой модели, инференс с доверительными интервалами, интерпретация вкладов и подбор сопоставимых объектов, генерация PDF-отчёта. Архитектура модульная и масштабируемая: ML-сервис (realval), backend-шлюз (go_back) и веб-интерфейс (ui), взаимодействие по REST, хранилища на витрине и PostGIS.

Тестирование (функциональное, интеграционное, системное, нагрузочное) подтвердило корректность ключевых сценариев: геокодирование, расчёт оценки, объяснения, компараблы и отчёты. Зафиксированы метрики качества (MAE, RMSE, MAPE по лог-таргету, покрытие и ширина интервалов) и эксплуатационные показатели (латентность p50/p95).

Перспективы развития: расширение геослоёв и витрины, онлайн-мониторинг качества, автоматическое переобучение, CI/CD, телеметрия, а также поддержка сценариев продажи. Решение соответствует ТЗ и готово к демонстрации.

ЗАКЛЮЧЕНИЕ

Целью данной выпускной квалификационной работы была разработка интеллектуальной системы анализа недвижимости, предназначенной для оценки рыночной ставки аренды квартиры с выводом доверительного интервала, объяснением влияющих факторов и подбором сопоставимых объектов. В ходе работы были решены следующие задачи:

- выполнен анализ предметной области и существующих отечественных и зарубежных решений;
- сформулированы и уточнены требования к системе, подготовлено и утверждено техническое задание;
- собран и подготовлен датасет из объявлений об аренде и внешних геоданных, проведены очистка, геокодирование и построение геопризнаков;
- разработаны и обучены модели машинного обучения (градиентный бустинг и табличная нейронная сеть) с пространственно-временной валидацией;
- реализованы методы оценки неопределённости (квантильный и конформный подходы) и интерпретации вкладов признаков;
- создан механизм поиска сопоставимых объектов по атрибутам и локализации; спроектирована архитектура и реализован веб-сервис с REST-API и пользовательским интерфейсом;
- проведено тестирование, оценена надёжность программного обеспечения и подготовлено руководство пользователя.

Теоретическая значимость работы заключается в обосновании и описании комплексного подхода к задаче оценки аренды недвижимости, включающего подготовку геообогащённой витрины признаков, пространственно-временную схему валидации, интервальные предсказания и интерпретируемость результатов. Также описаны варианты построения доверительных интервалов на

основе квантильной регрессии и конформного предсказания, а также использование SHAP-подхода для формирования локальных объяснений.

Практическая значимость работы состоит в создании сервиса RentAdvisor, который позволяет по введённым параметрам квартиры получить оценку арендной ставки, доверительный интервал и список сопоставимых предложений. Наличие объяснений факторов и компараблов делает результат более понятным для пользователя и удобным для предварительного анализа.

Перспективы развития системы включают расширение набора поддерживаемых городов и сегментов рынка, добавление новых источников данных и геопризнаков, развитие мониторинга качества и процедур переобучения моделей, а также расширение интерфейсов интеграции через API. Возможным направлением является поддержка дополнительных сценариев использования и форматов отчётности.

Все задачи, поставленные перед ВКР, выполнены.

The aim of this final qualification work was to develop an intelligent real estate analysis system designed to estimate the market rental rate of an apartment, provide a confidence interval, explain the influencing factors, and select comparable properties. During the work, the following tasks were completed:

- an analysis of the subject area and existing Russian and foreign solutions was carried out;
- the system requirements were formulated and refined, and the technical specification was prepared and approved;
- a dataset based on rental listings and external geodata was collected and prepared; data cleaning, geocoding, and geospatial feature generation were performed;
- machine learning models, including gradient boosting and a tabular neural network, were developed and trained using spatial-temporal validation;

- uncertainty estimation methods, including quantile and conformal approaches, as well as feature contribution interpretation methods, were implemented;
- a mechanism for searching comparable properties by attributes and location was created; the system architecture was designed, and a web service with a REST API and user interface was implemented;
- testing was carried out, the reliability of the software was assessed, and the user manual was prepared.

The theoretical significance of the work lies in the justification and description of an integrated approach to the task of rental property valuation. This approach includes the preparation of a geo-enriched feature store, a spatial-temporal validation scheme, interval predictions, and interpretable results. The work also describes methods for constructing confidence intervals based on quantile regression and conformal prediction, as well as the use of the SHAP approach to generate local explanations.

The practical significance of the work consists in the creation of the RentAdvisor service, which allows users to obtain a rental rate estimate, a confidence interval, and a list of comparable offers based on the entered apartment parameters. The presence of factor explanations and comparable properties makes the result more understandable for the user and more convenient for preliminary analysis.

Further development of the system may include expanding the list of supported cities and market segments, adding new data sources and geospatial features, improving quality monitoring and model retraining procedures, as well as extending API-based integration interfaces. Another possible direction is support for additional use cases and reporting formats.

All tasks set for the final qualification work have been completed.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Банк России. Ипотечное жилищное кредитование. Показатели рынка [Электронный ресурс]. – URL: https://www.cbr.ru/statistics/bank_sector/mortgage/Indicator_mortgage/0825/ (дата обращения: 20.03.2026).
2. ДОМ.РФ. Предложение арендного жилья в России не успевает за растущим спросом [Электронный ресурс]. – URL: <https://аренда.дом.рф/news/dom-rf-predlozhenie-arendnogo-zhilya-v-rossii-ne-uspevaet-za-rastushchim-sprosom/> (дата обращения: 20.03.2026).
3. Яндекс Недвижимость. Аренда квартир в Москве [Электронный ресурс]. – URL: <https://realty.yandex.ru/moskva/> (дата обращения: 20.03.2026).
4. ЦИАН. Аналитика рынка недвижимости [Электронный ресурс]. – URL: <https://www.cian.ru/analitika-nedvizhimosti/> (дата обращения: 20.03.2026).
5. Авито Недвижимость. Раздел «Квартиры в аренду» [Электронный ресурс]. – URL: <https://www.avito.ru/rossiya/kvartiry/sdam> (дата обращения: 20.03.2026).
6. ГОСТ 34.602–2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание (модернизацию) автоматизированной системы [Электронный ресурс]. – URL: <https://docs.cntd.ru/document/1200172749> (дата обращения: 20.03.2026).
7. ГОСТ Р ИСО/МЭК 12207–2010. Системная и программная инженерия. Процессы жизненного цикла программных средств [Электронный ресурс]. – URL: <https://docs.cntd.ru/document/1200082859> (дата обращения: 20.03.2026).

8. Баскина А. Е. Современные методы оценки стоимости объектов недвижимости на российском рынке [Электронный ресурс]. – URL: <https://kldjournal.ru/jour/article/view/2392> (дата обращения: 20.03.2026).
9. Астраханцева Е. Н., Пожидаева Н. А. Применение методов машинного обучения в оценке коммерческой недвижимости [Электронный ресурс]. – URL: <https://cyberleninka.ru/article/n/primenenie-metodov-mashinnogo-obucheniya-v-otsenke-kommercheskoy-nedvizhimosti> (дата обращения: 20.03.2026).
10. Родионов А. В. Методы машинного обучения в исследовании рынка жилой недвижимости [Электронный ресурс]. – URL: <https://cyberleninka.ru/article/n/metody-mashinnogo-obucheniya-v-issledovanii-rynka-zhiloy-nedvizhimosti> (дата обращения: 20.03.2026).
11. Лейфер И. Д. Модели массовой оценки недвижимости с использованием методов машинного обучения [Электронный ресурс]. – URL: <https://cyberleninka.ru/article/n/modeli-massovoy-otsenki-nedvizhimosti-s-ispolzovaniem-metodov-mashinnogo-obucheniya> (дата обращения: 20.03.2026).
12. Roberts D. R. et al. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure [Электронный ресурс]. – *Ecography*, 2017. – URL: <https://onlinelibrary.wiley.com/doi/10.1111/ecog.02881> (дата обращения: 20.03.2026).
13. Valavi R. et al. Block cross-validation for spatially structured data [Электронный ресурс]. – *Methods in Ecology and Evolution*, 2019. – URL: <https://besjournals.onlinelibrary.wiley.com/doi/full/10.1111/2041-210X.13107> (дата обращения: 20.03.2026).
14. Angelopoulos A. N., Bates S. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification [Электронный ресурс]. – 2021. – URL: <https://arxiv.org/abs/2107.07511> (дата обращения: 20.03.2026).

15. Dijk G. An Introduction to Conformal Prediction [Электронный ресурс]. – URL: <https://gdijcks.github.io/posts/conformal-intro/> (дата обращения: 20.03.2026).
16. Quantile regression for machine learning [Электронный ресурс]. – Ethen's Blog. – URL: https://ethen8181.github.io/machine-learning/ab_tests/quantile_regression/quantile_regression.html (дата обращения: 20.03.2026).
17. MAPIE Documentation. Model-Agnostic Prediction Interval Estimation [Электронный ресурс]. – URL: <https://mapie.readthedocs.io/en/latest/> (дата обращения: 20.03.2026).
18. LightGBM Documentation. Light Gradient Boosting Machine [Электронный ресурс]. – URL: <https://lightgbm.readthedocs.io/en/latest/> (дата обращения: 20.03.2026).
19. CatBoost Documentation. Gradient Boosting on Decision Trees [Электронный ресурс]. – URL: <https://catboost.ai/en/docs/> (дата обращения: 20.03.2026).
20. XGBoost Documentation. Scalable and Flexible Gradient Boosting [Электронный ресурс]. – URL: <https://xgboost.readthedocs.io/en/stable/> (дата обращения: 20.03.2026).
21. Lundberg S. M., Lee S.-I. A Unified Approach to Interpreting Model Predictions [Электронный ресурс]. – Advances in Neural Information Processing Systems, 2017. – URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (дата обращения: 20.03.2026).
22. SHAP Documentation. SHapley Additive exPlanations [Электронный ресурс]. – URL: <https://shap.readthedocs.io/en/latest/> (дата обращения: 20.03.2026).
23. Зотова Н. В., и др. Объяснимый искусственный интеллект: обзор методов интерпретации моделей [Электронный ресурс]. – URL:

- <https://cyberleninka.ru/article/n/ob-yasnimyy-iskusstvennyy-intellekt-obzor-metodov> (дата обращения: 20.03.2026).
24. Python Software Foundation. Python 3.12 Documentation [Электронный ресурс]. – URL: <https://docs.python.org/3/> (дата обращения: 20.03.2026).
25. The Go Programming Language. Go 1.x Documentation [Электронный ресурс]. – URL: <https://go.dev/doc/> (дата обращения: 20.03.2026).
26. FastAPI Documentation. Modern, fast (high-performance) web framework for building APIs with Python [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 20.03.2026).
27. PostGIS Documentation. Spatial and Geographic Objects for PostgreSQL [Электронный ресурс]. – URL: <https://postgis.net/documentation/> (дата обращения: 20.03.2026).
28. H3: A Hexagonal Hierarchical Geospatial Indexing System [Электронный ресурс]. – URL: <https://h3geo.org/docs/> (дата обращения: 20.03.2026).
29. React Documentation. A JavaScript library for building user interfaces [Электронный ресурс]. – URL: <https://react.dev/> (дата обращения: 20.03.2026).
30. Next.js Documentation. The React Framework for the Web [Электронный ресурс]. – URL: <https://nextjs.org/docs> (дата обращения: 20.03.2026).
31. Tailwind CSS Documentation. A utility-first CSS framework [Электронный ресурс]. – URL: <https://tailwindcss.com/docs> (дата обращения: 20.03.2026).
32. Kleiner E. Метрики качества моделей машинного обучения: MAE, RMSE, MAPE и другие [Электронный ресурс]. – Sky.pro, 2023. – URL: <https://sky.pro/wiki/python/metriki-kachestva-mashinnogo-obucheniya/> (дата обращения: 20.03.2026).

ПРИЛОЖЕНИЯ

Приложение А – Организационно-экономическое обоснование выпускной квалификационной работы.

Приложение Б – Графический материал (презентация) ВКР.

Приложение В — Программный код.

Приложение А

Организационно-экономическое обоснование выпускной квалификационной работы

Организационно-экономическое обоснование выпускной квалификационной работы формализуется как задача подготовки к реализации ИТ-проекта и начинается с определения его участников.

В проекте участвуют:

1. Руководитель – владелец автоматизируемого бизнес-процесса. В рассматриваемом бизнес-процессе им является руководитель направления оценки и аналитики недвижимости.
2. Руководитель ВКР бакалавра.
3. Студент-выпускник, объединяющий следующие роли: аналитик, проектировщик, разработчик, технический писатель, тестировщик. Роли, выполняемые студентом, определяются темой ВКР совместно с руководителем ВКР.

Основные участники проекта выполняют в нём следующие функции:

1. Владелец автоматизируемого бизнес-процесса: руководитель направления оценки и аналитики недвижимости, следит за процессом проектирования интеллектуальной системы анализа недвижимости. Участником проекта может быть лицо, занимающее руководящую должность в организации, которая является объектом исследования по теме ВКР и для которой предназначен результат проекта, над которым работает студент.
2. Руководитель ВКР – это сотрудник РТУ МИРЭА, который организует, планирует, оценивает и корректирует деятельность студента, оценивает получаемые им результаты, а также контролирует отдельные этапы работы, вносит необходимые коррективы и оценивает выполненную работу в целом, выступает в роли тимлида.

3. Студент – в процессе проектирования ИТ-решения выполняет роли аналитика, проектировщика, разработчика, тестировщика и технического писателя:

- в роли аналитика – систематизирует требования к интеллектуальной системе анализа недвижимости, выполняет постановку задач, анализирует предметную область оценки стоимости аренды квартир и обеспечивает соответствие получаемых результатов поставленным требованиям;
- в роли проектировщика – выполняет проектирование архитектуры ИТ-решения, состава модулей, пользовательского интерфейса, программных интерфейсов, структуры отчётов и логики взаимодействия компонентов системы;
- в роли разработчика – реализует программные компоненты системы: ML-сервис, backend-шлюз, веб-интерфейс, обработку запросов, формирование прогноза, доверительного интервала, объяснений и подбор сопоставимых объектов;
- в роли тестировщика – проверяет корректность работы основных сценариев: геокодирование, расчёт стоимости аренды, формирование интервала, подбор компараблов, генерацию отчёта и обработку ошибок ввода;
- в роли технического писателя – выполняет разработку технической и пользовательской документации, сопровождающей ИТ-решение.

Организация работ осуществляется на базе календарного планирования, которое включает в себя:

- планирование содержания проекта и декомпозицию работ;
- определение последовательности работ;
- планирование сроков, длительностей и логических связей работ;
- построение диаграммы Ганта.

Состав работ планируется в соответствии с ГОСТ Р 59793-2021 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания». Выбираемый стандарт для определения этапов и работ по проекту определяется темой ВКР, то есть, по сути, тем, что будет являться результатом. В соответствии с этим в состав этапов и работ, представленных в Таблице А.1, могут быть внесены коррективы по согласованию с руководителем ВКР.

В Таблице А.1 приведён состав этапов проектирования ИТ-решения, планирование сроков выполняется с учетом:

- шестидневной рабочей (учебной) недели руководителя ВКР и студента;
- выходных и праздничных дней;
- состава участвующих в разработке исполнителей.

Таблица А.1 – Календарный план выполнения проекта

Этап	Дата начала	Дата окончания	Кол-во раб. дней	Исполнители
1 Формирование требований к проекту				
1.1 Обследование предметной области и обоснование необходимости создания проекта	09.02.2026	10.02.2026	2	Руководитель направления оценки и аналитики недвижимости; Руководитель ВКР Разработчик проекта
1.2 Формирование требований пользователя к проекту	11.02.2026	11.02.2026	1	Руководитель направления оценки и аналитики недвижимости; Руководитель ВКР Разработчик проекта
1.3 Оформление отчета о выполненной работе	12.02.2026	12.02.2026	1	Разработчик проекта
2. Разработка концепции проекта				
2.1 Изучение объекта автоматизации и сценариев оценки недвижимости	13.02.2026	14.02.2026	2	Разработчик проекта
2.2 Проведение необходимых исследовательских работ	16.02.2026	17.02.2026	2	Разработчик проекта

Продолжение Таблицы А.1

2.3 Разработка вариантов концепции ИТ-решения, выбор и согласование варианта, удовлетворяющего требованиям	18.02.2026	19.02.2026	2	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
2.4 Оценка рисков проекта	20.02.2026	20.02.2026	1	Разработчик проекта
2.5 Оформление отчета о выполненной работе	21.02.2026	21.02.2026	1	Разработчик проекта
3. Техническое задание				
3.1 Разработка и утверждение технического задания на ИТ-решение	24.02.2026	25.02.2026	2	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
3.2. Разработка документации	26.02.2026	27.02.2026	2	Разработчик проекта
4. Эскизный (пилотный) проект				
4.1 Разработка предварительных проектных решений	28.02.2026	18.03.2026	16	Разработчик проекта
4.2 Разработка программного кода ИТ-решения	19.03.2026	09.04.2026	19	Разработчик проекта
4.3 Разработка документации на ИТ-решение и его части	10.04.2026	13.04.2026	4	Разработчик проекта
5. Технический проект				
5.1 Разработка итоговых проектных решений	15.04.2026	28.04.2026	12	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
5.2 Разработка итоговой архитектуры ИТ-решения	29.04.2026	14.05.2026	13	Разработчик проекта
5.3 Доработка программного кода ИТ-решения	15.05.2026	30.05.2026	14	Разработчик проекта
5.4 Разработка документации на ИТ-решение и его части	01.06.2026	04.06.2026	4	Разработчик проекта
6. Рабочая документация				
6.1 Разработка рабочей документации на ИТ-решение	05.06.2026	11.06.2026	6	Разработчик проекта
6.2 Формирование комплекта рабочей документации на ИТ-решение	13.06.2026	17.06.2026	4	Разработчик проекта

Продолжение Таблицы А.1

6.3 Расчёт затрат на проведение работ	18.06.2026	23.06.2026	5	Разработчик проекта
7. Ввод в действие				
7.1 Подготовка объекта автоматизации к вводу ИТ-решения в действие	24.06.2026	26.06.2026	3	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
7.2 Подготовка пользовательской инструкции	27.06.2026	29.06.2026	2	Разработчик проекта
7.3 Комплектация ИТ-решения поставляемыми изделиями	30.06.2026	01.07.2026	2	Разработчик проекта
7.4 Развертывание ИТ-решения	02.07.2026	04.07.2026	3	Разработчик проекта
7.5 Тестирование ИТ-решения	06.07.2026	08.07.2026	3	Разработчик проекта
7.6 Ввод ИТ-решения в эксплуатацию	09.07.2026	14.07.2026	5	Разработчик проекта
7.7 Проведение приемочных испытаний	15.07.2026	18.07.2026	4	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
7.8 Сопровождение апробации ИТ-решения	21.07.2026	28.07.2026	7	Разработчик проекта

Таким образом, на разработку отводится 142 рабочих дня.

Диаграмма Ганта широко используется для визуализации хода выполнения задач, планирования ресурсов, графика рабочего времени и других данных, которые представляются набором временных интервалов. Она необходима для соблюдения сроков выполнения работ, и, соответственно, анализа хода проектирования ИТ-решения.

Календарный план-график (диаграмма Ганта), разработанный в соответствии с данными из Таблицы А.1, представлен на Рисунках А.1 и А.2.

Рисунок А.1 отражает этапы и работы в рамках проекта, участников на каждом этапе, дату начала и завершения, а также длительность в часах как каждой работы, так и этапа.

Рисунок А.2 отражает непосредственно саму диаграмму Ганта с учетом последовательного выполнения работ.

Этап	Дата начала	Дата окончания	Кол-во раб. дней	Длительность, ч.	Исполнители
1 Формирование требований к проекту					
1.1 Обследование предметной области и обоснование необходимости создания проекта	09.02.2026	10.02.2026	2	16	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
1.2 Формирование требований пользователя к проекту	11.02.2026	11.02.2026	1	8	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
1.3 Оформление отчета о выполненной работе	12.02.2026	12.02.2026	1	8	Разработчик проекта
2 Разработка концепции проекта					
2.1 Изучение объекта автоматизации и сценариев оценки недвижимости	13.02.2026	14.02.2026	2	16	Разработчик проекта
2.2 Проведение необходимых исследовательских работ	16.02.2026	17.02.2026	2	16	Разработчик проекта
2.3 Разработка вариантов концепции ИТ-решения, выбор и согласование варианта, удовлетворяющего требованиям	18.02.2026	19.02.2026	2	16	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
2.4 Оценка рисков проекта	20.02.2026	20.02.2026	1	8	Разработчик проекта
2.5 Оформление отчета о выполненной работе	21.02.2026	21.02.2026	1	8	Разработчик проекта
3 Техническое задание					
3.1 Разработка и утверждение технического задания на ИТ-решение	24.02.2026	25.02.2026	2	16	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
3.2 Разработка документации	26.02.2026	27.02.2026	2	16	Разработчик проекта
4 Эскизный (пилотный) проект					
4.1 Разработка предварительных проектных решений	28.02.2026	18.03.2026	16	128	Разработчик проекта
4.2 Разработка программного кода ИТ-решения	19.03.2026	09.04.2026	19	152	Разработчик проекта
4.3 Разработка документации на ИТ-решение и его части	10.04.2026	14.04.2026	4	32	Разработчик проекта
5 Технический проект					
5.1 Разработка итоговых проектных решений	15.04.2026	28.04.2026	12	96	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
5.2 Разработка итоговой архитектуры ИТ-решения	29.04.2026	14.05.2026	13	104	Разработчик проекта
5.3 Доработка программного кода ИТ-решения	15.05.2026	30.05.2026	14	112	Разработчик проекта
5.4 Разработка документации на ИТ-решение и его части	01.06.2026	04.06.2026	4	32	Разработчик проекта
6 Рабочая документация					
6.1 Разработка рабочей документации на ИТ-решение	05.06.2026	11.06.2026	6	48	Разработчик проекта
6.2 Формирование комплекта рабочей документации на ИТ-решение	13.06.2026	17.06.2026	4	32	Разработчик проекта
6.3 Расчёт затрат на проведение работ	18.06.2026	23.06.2026	5	40	Руководитель ВКР Разработчик проекта
7 Ввод в действие					
7.1 Подготовка объекта автоматизации к вводу ИТ-решения в действие	24.06.2026	26.06.2026	3	24	Руководитель направления оценки и аналитики недвижимости Разработчик проекта
7.2 Подготовка пользовательской инструкции	27.06.2026	29.06.2026	2	16	Разработчик проекта
7.3 Комплектация ИТ-решения поставляемыми артефактами	30.06.2026	01.07.2026	2	16	Разработчик проекта
7.4 Развертывание ИТ-решения	02.07.2026	04.07.2026	3	24	Разработчик проекта
7.5 Тестирование ИТ-решения	06.07.2026	08.07.2026	3	24	Разработчик проекта
7.6 Ввод ИТ-решения в опытную эксплуатацию	09.07.2026	14.07.2026	5	40	Разработчик проекта
7.7 Проведение приемочных испытаний	15.07.2026	18.07.2026	4	32	Руководитель направления оценки и аналитики недвижимости Руководитель ВКР Разработчик проекта
7.8 Сопровождение апробации ИТ-решения	21.07.2026	28.07.2026	7	56	Разработчик проекта
Итого рабочих дней			142	1136	

Рисунок А.1 – Календарный план-график (Часть 1)

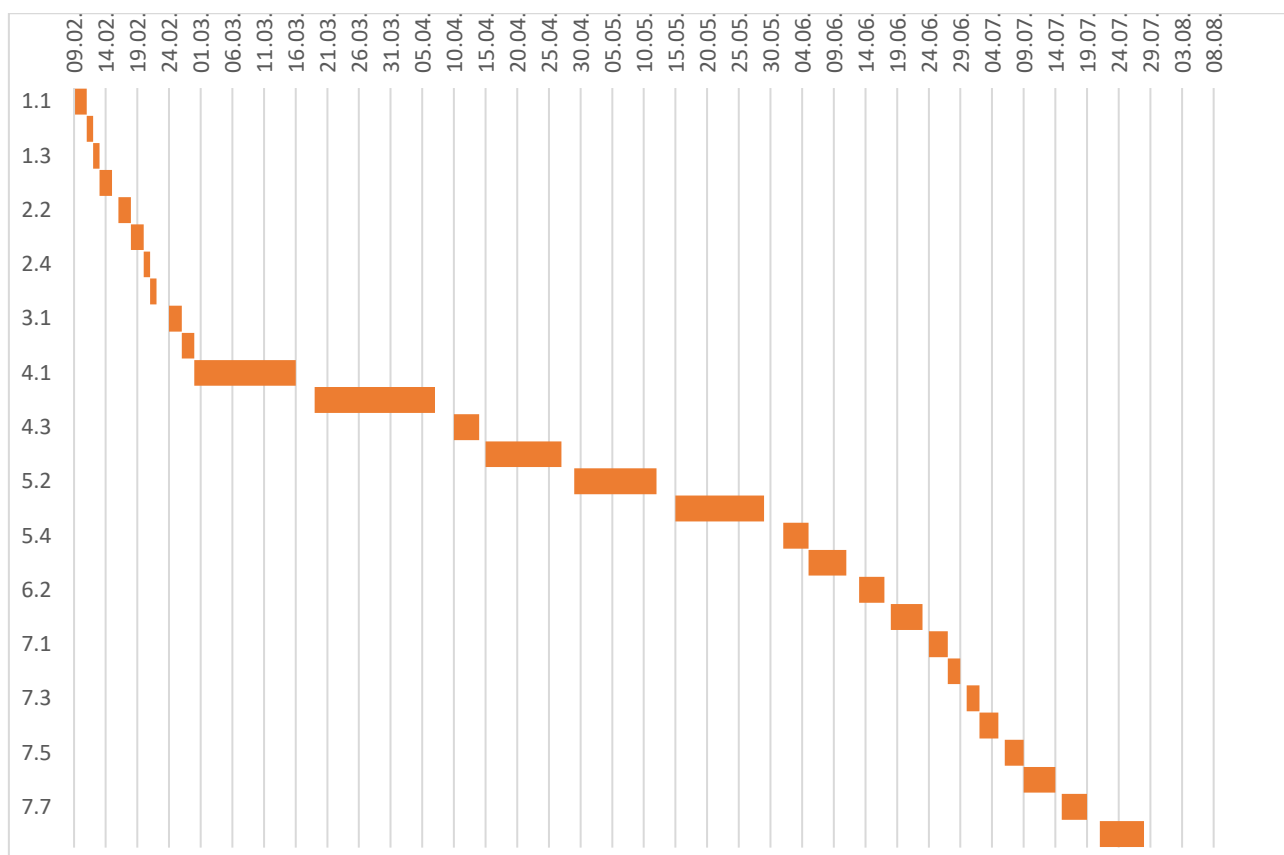


Рисунок А.1 – Календарный план-график (Часть 2)

После организации и планировании работ, а именно построения календарного плана далее рассмотрен расчет затрат на проведение работ.

Себестоимость проектирования ИТ-решения складывается из затрат по следующим статьям:

- основная заработная плата;
- дополнительная заработная плата;
- страховые взносы — 30% от фонда оплаты труда (ФОТ), а также 0,2% — ставка за травматизм;
- материалы, покупные изделия и полуфабрикаты, а также до 20% от соответствующей суммы на транспортно-заготовительные расходы (ТЗР);
- расходы на приобретение компьютерной техники и программного обеспечения, а также до 20% от соответствующей суммы на ТЗР;

- амортизационные отчисления по используемому в проекте оборудованию;
- эксплуатационные расходы, связанные с использованием компьютерной техники;
- накладные расходы — до 150% от основной заработной платы.

Результат анализа размера оплаты труда в регионе расположения объекта исследования: город Москва, представлен в Таблице А.2.

Согласно Производственному календарю на 2026 год, годовой фонд рабочего времени при шестидневной рабочей неделе составит 299 рабочих дней, при пятидневной рабочей неделе составит 247 рабочих дней.

Таблица А.2 – Размер оплаты труда в регионе по должностям

Исполнитель (должность)	Ряд значений	Источник данных	Медианное значение оплаты труда, руб.
Руководитель направления оценки и аналитики недвижимости	100 000	https://hh.ru/search/vacancy?text=руководитель+аналитики+недвижимости	130 000
	110 000	https://www.superjob.ru/vacancy/search/?keywords=руководитель%20аналитики	
	130 000	https://moskva.gorodrabot.ru/руководитель_аналитики	
	155 000	https://hh.ru/search/vacancy?text=руководитель+направления+аналитики	
	172 000	https://hh.ru/search/vacancy?text=руководитель+оценки+недвижимости	
Руководитель ВКР	90 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	110 000
	95 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	
	110 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	
	120 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	
	125 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	
Разработчик проекта	60 000	https://hh.ru/search/vacancy?text=разработчик+python+backend+junior	100 000
	80 000	https://hh.ru/search/vacancy?text=backend+разработчик+стажер	
	100 000	https://hh.ru/search/vacancy?text=python+разработчик	
	120 000	https://hh.ru/search/vacancy?text=go+backend+разработчик	
	125 000	https://hh.ru/search/vacancy?text=разработчик+проекта	

Дневная тарифная ставка (ТС) по каждому участнику может быть рассчитана следующим образом:

$$ТС = \frac{ОК \times 12}{НРВ}, \quad (A.1)$$

где ТС – дневная тарифная ставка, руб. в день;

ОК – месячный оклад участника проекта, руб;

12 – число месяцев в году, руб;

НРВ – годовой фонд рабочего времени, в дн.

В Таблице А.3 приведен расчет заработной платы участников проекта. Количество рабочих дней для каждой роли определяется на основе Таблицы А.1 с учетом участия в выполнении работ.

Таблица А.3 – Расчет основной заработной платы

Наименование этапа	Исполнитель (должность)	Месячный оклад, руб.	Оплата за день, руб.	Количество рабочих дней	Оплата за этап, руб.
1. Формирование требований к проекту	Руководитель направления оценки и аналитики недвижимости	130 000	5 200	3	15 600
	Руководитель ВКР	110 000	4 400	3	13 200
	Разработчик проекта	100 000	4 000	4	16 000
2. Разработка концепции проекта	Руководитель направления оценки и аналитики недвижимости	130 000	5 200	2	10 400
	Руководитель ВКР	110 000	4 400	2	8 800
	Разработчик проекта	100 000	4 000	8	32 000
3. Техническое задание	Руководитель направления оценки и аналитики недвижимости	130 000	5 200	2	10 400
	Разработчик проекта	100 000	4 000	4	16 000
4. Эскизный проект	Разработчик проекта	100 000	4 000	39	156 000

Продолжение Таблицы А.3

5. Технический проект	Руководитель направления оценки и аналитики недвижимости	130 000	5 200	12	62 400
	Разработчик проекта	100 000	4 000	43	172 000
6. Рабочая документация	Разработчик проекта	100 000	4 000	15	60 000
	Руководитель ВКР	110 000	4 400	5	22 000
7. Ввод в действие	Руководитель направления оценки и аналитики недвижимости	130 000	5 200	7	36 400
	Руководитель ВКР	110 000	4 400	4	17 600
	Разработчик проекта	100 000	4 000	29	116 000
Итого	764 800				

Таким образом, заработная плата исполнителей составит 764 800 рублей.

В среднем расходы по дополнительной заработной плате составляют 20-30% от суммы основной заработной платы. В процессе определения сметы затрат вводится понятие «фонд оплаты труда», представляющее собой сумму основной (ОЗП) и дополнительной заработной платы (ДЗП). Фонд оплаты труда (ФОТ) используется при расчете взносов в социальный фонд. Во всех других случаях (накладные расходы, командировки и др.) расчеты ведутся от базы основной заработной платы.

Расходы на дополнительные заработные платы участников проекта представлены в Таблице А4.

Таблица А.4 – Расчет дополнительной заработной платы

Показатель	Процент, %	Значение (руб.)
Основная заработная плата (ОЗП)	-	764 800
Дополнительная заработная плата (ДЗП)	20	152 960
Фонд оплаты труда (ФОТ)	-	917 760

Таким образом, фонд оплаты труда составляет 917 760 рублей.

В соответствии со статьей 425 НК РФ в 2026 году для лиц, которые производят выплаты и вознаграждения физическим лицам (за исключением плательщиков, для которых установлены пониженные тарифы страховых взносов), установлен Единый тариф страховых взносов 30 % по направлениям:

- обязательное пенсионное страхование (ОПС),
- обязательное социальное страхование на случай временной нетрудоспособности и в связи с материнством (ОСС),
- обязательное медицинское страхование (ОМС).

Таким образом, страховые взносы (СВ) во внебюджетные фонды составят 30,2 % от фонда оплаты труда (ФОТ), который состоит из основной и дополнительной заработной платы, и равны 277 164 рубля.

В процессе проектирования и разработки ИТ-решения также формируются затраты, связанные с приобретением материалов, покупных изделий, комплектующих изделий и других материальных ценностей, расходуемых непосредственно.

Стоимость материалов представлена в Таблице А.5.

Таблица А.5 – Расчет стоимости материалов

Наименование материалов	Единицы измерения	Количество	Цена за единицу, руб.	Стоимость, руб.
Флеш-накопитель	шт.	1	550	550
Бумага А4	пачка	1	438	438
Картридж для принтера	шт.	1	4 150	4 150
Канцелярский набор	шт.	1	577	577
Итого материалов				5 715,0
Транспортно-заготовительные расходы (10%)				571,5
Итого:				6 286,5

Таким образом, сумма затрат по материалам составляет 6 287 руб.

Затраты на комплектующие и изготовление программно-аппаратного комплекса представлены в Таблице А.6.

Таблица А.6 – Расчет стоимости комплектующих

Наименование элементов	Единицы измерения	Количество	Цена за единицу (руб.)	Стоимость (руб.)
Комплектующие не приобретаются	-	0	0	0
Итого комплектующих				0
Транспортно-заготовительные расходы (10 %)				0
Итого:				0

Таким образом, стоимость комплектующих составляет 0 руб. А в целом по статье «Материалы, покупные изделия и полуфабрикаты» с учетом ТЗР — 6 287 руб.

Расходы на приобретение компьютерной техники и программного обеспечения включают в себя стоимость вычислительной техники и программного обеспечения, необходимых как для проектирования и разработки ИТ-решения, так и для последующей его эксплуатации.

В состав технического обеспечения входят средства компьютерной, организационной и коммуникационной техники.

В соответствии с требуемым техническим обеспечением к нему относится: рабочая станция разработчика, сервер приложений системы, сервер под реляционное хранилище данных и принтер.

Так как ИТ-инфраструктура для проектирования и разработки ИТ-решения уже есть, то затраты по приобретению оборудования будут отсутствовать.

Амортизационные отчисления по используемому в проекте оборудованию определяются на основе балансовой стоимости основных фондов, а также с учетом установленных норм. Начисление амортизации происходит на основные фонды стоимостью от 100 тыс. руб. и выше, если иное не уставлено в учётной политики организации. Способ начисления амортизации также определяются

учетной политикой организации. Предполагается, что установлен линейный способ начисления амортизации.

Амортизационные отчисления определяются по формуле:

$$A = \frac{C}{T}, \quad (A.2)$$

где А – сумма амортизационных отчислений за месяц, руб.;

С – первоначальная стоимость объекта, руб.;

Т – срок полезного использования, мес.

В Таблице А.7 представлены расчеты амортизационных отчислений по использованному во время разработки проекта оборудованию.

Таблица А.7 – Амортизационные отчисления

Наименование оборудования	Первоначальная стоимость объекта, руб.	Срок полезного использования, мес.	Месячная сумма амортизации, руб.	Период эксплуатации, мес.	Сумма, руб.
Сервер приложений и ML-сервиса	400 000,00	36,00	11 111,11	3,50	38 888,89
Сервер под PostgreSQL/PostGIS	206 000,00	36,00	5 722,22	3,50	20 027,78
Рабочая станция разработчика	120 000,00	36,00	3 333,33	3,50	11 666,67
МФУ	107 000,00	60,00	1 783,33	3,50	6 241,67
Итого:					76 825,00

Таким образом, сумма амортизационных отчислений составит 75 365 руб.

Из техники в работе могут использоваться компьютеры, серверное оборудование и принтеры разных моделей с различным энергопотреблением. Компьютер используется на протяжении всей работы над проектом, и, согласно Таблице А.1, это время составляет 1 136 часов (142 дня по 8 часов в день на одну

единицу техники), в то время как время работы принтера будет существенно меньше. Стоимость 1 кВт электроэнергии в Москве равна 8 руб. 00 коп. Расчет эксплуатационных расходов представлен в Таблице А.8.

Таблица А.8 – Расчет эксплуатационных расходов

Наименование техники	Мощность, кВт	Время использования, ч.	Стоимость 1 кВт, руб.	Расходы, руб.
Сервер	0,5	516	8	2064
Компьютер	0,41	1136		3726,08
МФУ	0,12	2		1,92
Коммутатор	0,1	516		412,8
Маршрутизатор	0,2	516		825,6
Примерные общие прочие эксплуатационные расходы:				7030,4

Таким образом, расчетные общие прочие эксплуатационные расходы составляют 7 030 рублей.

К накладным расходам (НР) относятся расходы на содержание и ремонт зданий, сооружений, оборудования, инвентаря. Это затраты, сопутствующие основному производству, но не связанные с ним напрямую, не входящие в стоимость труда и материалов.

В рамках реализации проекта это могут быть расходы, связанные с содержанием и арендой площадей, с приобретением расходных материалов для офиса, транспортными расходами, оплатой консультационных, информационных и клиринговых услуг, а также с командировочными расходами.

Сумма данных расходов определяется процентом от суммы основной заработной платы (ОЗП), в данном проекте они принимаются равными 100%.

$$\text{НР} = 1 \times 764\,800 = 764\,800 \text{ руб.}$$

Накладные расходы составят 764 800 руб.

Вышеперечисленные статьи затрат и результаты расчетов по ним обобщены в Таблицу А.9.

Полная стоимость проекта составляет 2 048 406 рублей.

Таблица А.8 – Полная себестоимость проекта

Номенклатура статей расходов	Расходы, руб.	Доля расходов, %
Основная заработная плата	764 800,00	37,31
Дополнительная заработная плата	152 960,00	7,46
Страховые взносы	277 163,52	13,52
Материалы, покупные изделия и полуфабрикаты	6 286,50	0,31
Расходы на приобретение компьютерной техники и программного обеспечения	0,00	0,00
Амортизационные отчисления	76 825,00	3,75
Эксплуатационные расходы, связанные с использованием компьютерной техники	7 030,40	0,34
Накладные расходы	764 800,00	37,31
Прочие	0,00	0,00
Итого:	2 049 865,42	100,00

Для визуализации долевого состава статей затрат в общей себестоимости проекта представлена круговая диаграмма – Рисунок А.3.

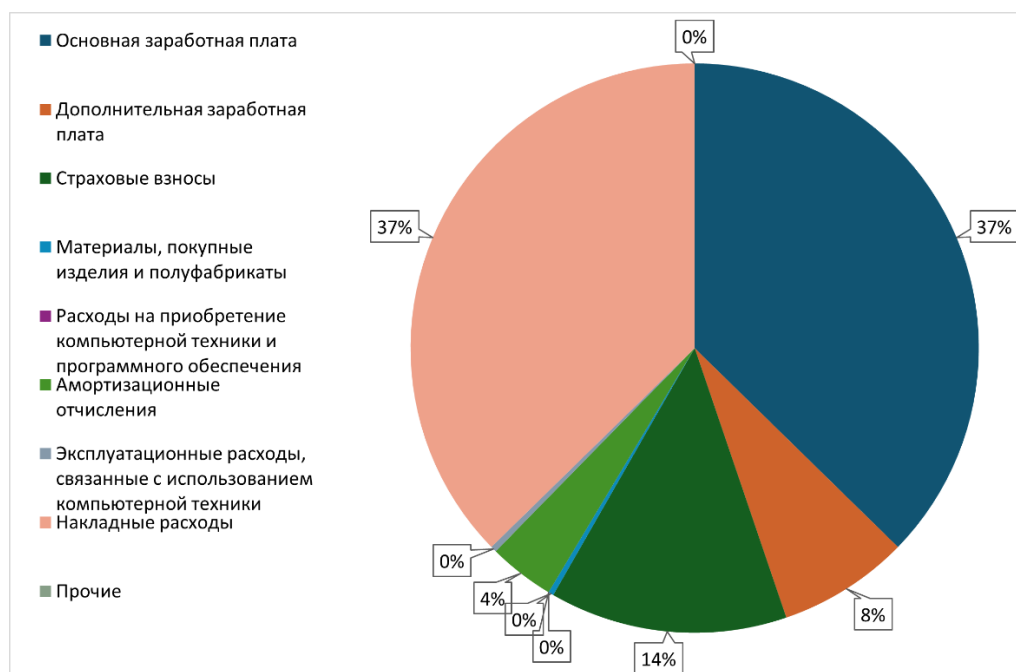


Рисунок А.3 – Структура затрат по проекту

Таким образом, в организационно-экономическом обосновании ВКР было реализовано планирование работ по проектированию и разработке ИТ-решения, а также выполнен расчет себестоимости соответствующего проекта. Полная стоимость проекта составляет 2 048 406 рублей.

Приложение Б

Графический материал (презентация) ВКР



Актуальность

- Рынок аренды неоднороден: цена сильно зависит от района, времени и состояния жилья.
- Пользователю сложно понять “справедливую” ставку → риск переплаты/ошибки решения.
- Существующие онлайн-оценки часто непрозрачны: дают одно число без интервала и объяснений.
- Нужен инструмент: прогноз + доверительный интервал + объяснение факторов + компараблы.

Объект, предмет, область применения

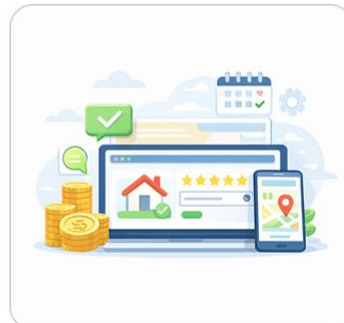
Объект
рынок аренды квартир



Предмет
прогноз арендной ставки
по параметрам жилья и
геоданным



Область
веб-сервис оценки
аренды



3

Цель и задачи

Цель

построить модель, предоставляющую прогноз с доверительным интервалом и подбором компараблов на основе методов машинного обучения и геопространственных признаков.

Задачи

1. Сбор и подготовка данных (объявления + геоданные), витрина признаков.
2. Обучение модели и модуля интервалов (квантильный/конформный).
3. Реализация объяснимости (SHAP) и модуля компараблов.
4. Реализация сервиса: ML-сервис + backend-шлюз + UI, отчёт PDF.

4

Анализ аналогов

Критерий	Яндекс.Недвижимость	ЦИАН	Моя система
Оценка по адресу	✓	✓	✓
Доверительный интервал / диапазон	✗	✓	✓
Объяснение факторов	✗	✗	✓
Подбор похожих объектов	✓	✓	✓
Отчёт / экспорт результата	✗	✗	✓
Прозрачность метода	✗	✗	✓ (через объяснения)
Публичный API	✗	✗	✓ (REST)
Ориентация на агентский сценарий / уточнение роли	✓ (“что хотим сделать с квартирой”)	✓ (вопрос: агент/много уточнений)	✗

Аналоги дают оценку и похожие объекты, но не дают объяснения причин и удобной выдачи (отчёт/API); поэтому разрабатывается система “прогноз + интервал + объяснения + компараблы”.

5

Данные и подготовка



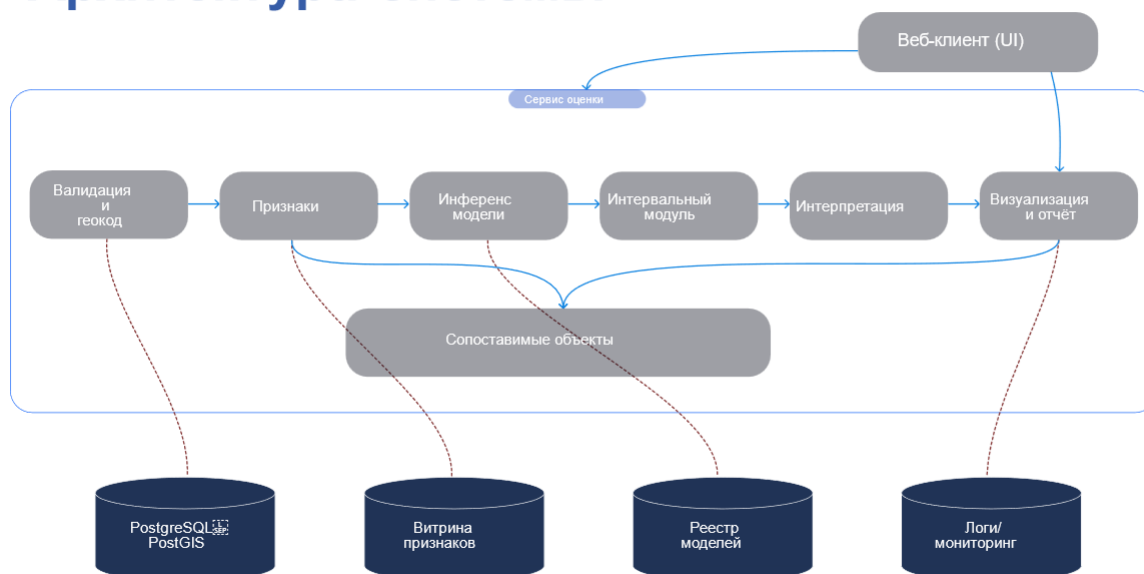
6

Методы и модель

- Модель: градиентный бустинг по табличным признакам (в т.ч. геопризнаки).
- Валидация: временное разбиение + контроль пространственного “подсматривания”.
- Интервалы: квантильный и/или конформный режим.
- Объяснения: SHAP (локально по объекту).

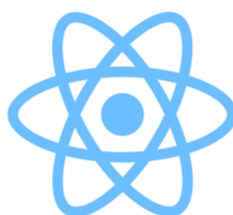
7

Архитектура системы



8

Используемые технологии



Обработка данных и ML:

- Python, pandas, NumPy
- LightGBM, CatBoost, XGBoost
- SHAP, MAPE

Инфраструктура и сервисы:

- FastAPI – REST-API ML-сервиса
- PostgreSQL + PostGIS – хранилище табличных и пространственных данных
- Docker – контейнеризация

Веб и интеграция:

- Go (Gin) – backend-шлюз
- React + Next.js, Tailwind CSS – веб-интерфейс
- GoFPDF – генерация HTML/PDF-отчётов

9

Результаты экспериментов

- MAPE $\approx 13.2\%$ $\rightarrow$ в среднем ошибка $\sim 15\%$
- ТЗ выполнено: точность $\geq 75\%$ (ошибка $\leq 25\%$)

Детали отчета

Результат оценки

Киевская улица, 16, Москва

Прогноз стоимости
147 534 Р
в месяц

Минимум диапазона
110 454 Р
в месяц

Максимум диапазона
197 063 Р
в месяц

Характеристики объекта

Комнат 2 комн.	Площадь 58 м²	Этаж 2 из 5
Год постройки 1935	Материал дома кирпич	

Анализ

Квартира в городе Москва, по адресу «Киевская улица, 16». Оценочная ставка долгосрочной аренды: около 147 534 Р в месяц (ожидаемый диапазон 110 454-197 063 Р).

Цена месячной аренды данной двухкомнатной квартиры площадью 58 м² прогнозируется моделью на уровне примерно 148 000 рублей. Интервал возможных значений колеблется между 110 000 и 197 000 рублями. Квартира расположена на Киевской улице, всего в трёх минутах пешком от станции метро «Студенческая», что существенно повышает её привлекательность для арендаторов. Однако год постройки дома (1935) и отсутствие ремонта снижают рыночную стоимость объекта. Дополнительное влияние оказывает удалённость от центра города, составляющее порядка 4,5 км, а также общая плотность застройки района.

Ключевые факторы:

- Удалённость от центра
- Удобство расположения рядом с метро
- Площадь квартиры
- Год постройки здания
- Отсутствие ремонта

Сопоставимые объекты

Студенческая (0.01 км)	2 комн., 58 м²	150 000 Р	🔗
Студенческая (0.44 км)	2 комн., 55 м²	80 000 Р	🔗
Студенческая (0.57 км)	2 комн., 50 м²	130 000 Р	🔗

10

Видео-демонстрация

The screenshot shows the 'RentAdvisor' web application. At the top, it says 'Нейросетевой прогноз стоимости аренды' (Neural network rental price forecast) and 'ML сервис: В сети' (ML service: Online). The main heading is 'Оценка рыночной стоимости аренды квартир в Москве' (Market rental price estimation in Moscow). Below this, a paragraph explains that the application uses modern machine learning algorithms for accurate forecasting, analyzing factors like location, area, and condition. Three boxes highlight features: 'Точный прогноз' (Accurate forecast), 'Диапазон уверенности' (Confidence interval), and 'Сопоставимые объекты' (Comparable objects). A section titled 'Оценка стоимости аренды' (Rental price estimation) contains a form with fields for address, city, number of rooms, total area, living area, kitchen area, floor, and building year. The form is currently empty, with placeholder text like 'Неизвестная улица, №1' and 'Москва'.

11

Заключение

- Реализован сервис оценки аренды: прогноз + интервал + объяснения + компараблы.
- Подготовлен датасет и витрина признаков, выполнено обучение и валидация модели.
- Реализована сервисная архитектура (UI + backend + ML) и генерация отчёта.
- Направления развития: расширение региона/данных, мониторинг качества, переобучение.

12

Достижения



13

Спасибо за внимание

Приложение В

Программный код

Листинг В.1 – ML-сервис оценки стоимости аренды

```
from __future__ import annotations

import json
import os
import subprocess
import tempfile
from pathlib import Path
from typing import Optional
import sys
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware # <-- добавить
from pydantic import BaseModel, Field

app = FastAPI(
    title="RealVal ML API",
    version="1.0.0",
    description="HTTP-обертка над realval CLI для оценки аренды",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # В продакшене укажите конкретные домены
    allow_credentials=True,
    allow_methods=["*"], # Или ["GET", "POST"]
    allow_headers=["*"],
)

# Где лежит модель (можно переопределить через ENV)
MODEL_PATH = os.getenv("REALVAL_MODEL_PATH",
    "models/artefacts/rent_lgbm.joblib")
# DSN для БД читается внутри CLI из POSTGRES_DSN_URL, тут его трогать не надо

class AddressRequest(BaseModel):
    city: str = Field("Москва", description="Город")
    address: str = Field(..., description="Адрес (улица, дом, корпус и т.п.)")
    rooms: int = Field(..., description="Количество комнат (0=студия, 1,2,3...)")
    area_total: float = Field(..., description="Общая площадь, м²")
    area_living: Optional[float] = Field(None, description="Жилая площадь, м²")
    area_kitchen: Optional[float] = Field(None, description="Площадь кухни, м²")
    floor: Optional[int] = Field(None, description="Этаж")
    floors_total: Optional[int] = Field(None, description="Этажность дома")
    year_built: Optional[int] = Field(None, description="Год постройки")
    house_material: Optional[str] = Field(None, description="Материал дома")
    condition: Optional[str] = Field(None, description="Состояние/ремонт")
    with_text: bool = Field(
        False,
        description="Генерировать текстовое объяснение через GigaChat",
    )

@app.get("/health")
def health_check():
```

Продолжение листинга В.1

```
"""Быстрая проверка работоспособности сервиса"""
return {
    "status": "ok",
    "service": "RealVal ML API",
    "model_path": MODEL_PATH,
    "model_exists": Path(MODEL_PATH).exists(),
}

@app.get("/health/deep")
def deep_health_check():
    """Проверка с загрузкой модели"""
    from joblib import load
    try:
        model_exists = Path(MODEL_PATH).exists()
        if model_exists:
            # Пробуем загрузить модель
            _ = load(MODEL_PATH)
            return {
                "status": "ok",
                "model_loaded": True,
                "model_path": MODEL_PATH,
            }
        else:
            return {
                "status": "error",
                "model_loaded": False,
                "model_path": MODEL_PATH,
                "error": "Model file not found",
            }
    except Exception as e:
        return {
            "status": "error",
            "model_loaded": False,
            "error": str(e),
        }

@app.post("/v1/predict/address")
def predict_address(req: AddressRequest):
    """
    HTTP-эндпоинт, который под капотом вызывает:
    python -m realval predict-address --model-path ... --params tmp.json [--with-text]
    и возвращает JSON-отчет как есть.
    """
    # 1) пишем тело запроса во временный JSON
    with tempfile.NamedTemporaryFile("w+", suffix=".json", delete=False,
encoding="utf-8") as f_in:
        # exclude with_text, он идёт флагом, а не в params
        params = req.model_dump(exclude={"with_text"})
        json.dump(params, f_in, ensure_ascii=False)
        tmp_in = f_in.name

    with tempfile.NamedTemporaryFile("w+", suffix=".json", delete=False,
encoding="utf-8") as f_out:
        tmp_out = f_out.name

    try:
        cmd = [
            sys.executable,
```



```
        "-m",
        "realval",
        "predict-address",
        "--model-path",
        MODEL_PATH,
        "--comps-path",
        "data/features/base.parquet",
        "--params",
        tmp_in,
        "--out",
        tmp_out,
    ]
    if req.with_text:
        cmd.append("--with-text")

    # subprocess запускает CLI, который уже знает всё про geocode/DB/LLM
    proc = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
    )

    if proc.returncode != 0:
        err_msg = proc.stderr.strip() or "realval CLI failed"
        raise HTTPException(status_code=500, detail=err_msg)

    # Читаем результат из файла
    try:
        with open(tmp_out, "r", encoding="utf-8") as f:
            data = json.load(f)
    except (FileNotFoundError, json.JSONDecodeError) as e:
        raise HTTPException(
            status_code=500,
            detail=f"Failed to read result: {str(e)}",
        )

    return data

finally:
    # подчистим временный файл
    try:
        os.unlink(tmp_in)
        os.unlink(tmp_out)
    except OSError:
        pass
```

```
@dataclass
class LLMConfig:
    """
    Конфигурация LLM-генерации текстовых блоков.
    Читается из .env, но можно вручную переопределить из кода.
    """
    enabled: bool = True
    model: str = os.getenv("GIGACHAT_MODEL", "GigaChat-Pro")
    scope: str = os.getenv("GIGACHAT_SCOPE", "GIGACHAT_API_PERS")
    credentials: Optional[str] = os.getenv("GIGACHAT_AUTH_KEY")
    # по умолчанию выключаем проверку сертификатов (для локальной разработки)
    verify_ssl_certs: bool = os.getenv("GIGACHAT_VERIFY_SSL", "false").lower()

in (
    "1",
    "true",
    "yes",
)

_llm_instance: Optional[GigaChat] = None

def _get_llm(cfg: Optional[LLMConfig] = None) -> GigaChat:
    """
    Лениво создаёт инстанс GigaChat для LangChain.
    """
    global _llm_instance
    if _llm_instance is not None:
        return _llm_instance

    cfg = cfg or LLMConfig()
    if not cfg.credentials:
        raise RuntimeError("GIGACHAT_AUTH_KEY не задан в окружении (.env)")

    _llm_instance = GigaChat(
        credentials=cfg.credentials,
        scope=cfg.scope,
        model=cfg.model,
        verify_ssl_certs=cfg.verify_ssl_certs,
```

```
)
    return _llm_instance

def _build_toon_payload(report: Dict[str, Any]) -> str:
    """
    Собираем компактный объект для LLM и кодируем его в TOON.
    Берём только то, что нужно для текста, чтобы не тратить лишние токены.
    """
    obj = report.get("object", {}) or {}
    enr = report.get("enriched", {}) or {}
    pricing_block = report.get("pricing", {}) or {}
    expl = report.get("explanation", {}) or {}
    comps: List[Dict[str, Any]] = report.get("comparables") or []

    pricing = {
        "prediction_rub": pricing_block.get("prediction_rub"),
        "interval_low_rub": pricing_block.get("interval_low_rub"),
        "interval_high_rub": pricing_block.get("interval_high_rub"),
        "currency": pricing_block.get("currency", "RUB"),
        "deal_type": pricing_block.get("deal_type", "rent_long"),
    }

    compact_comps = []
    for c in comps[:5]:
        compact_comps.append(
            {
                "price_rub": c.get("price_rub"),
                "rooms": c.get("rooms"),
                "area_total": c.get("area_total"),
                "floor": c.get("floor"),
                "floors_total": c.get("floors_total"),
                "year_built": c.get("year_built"),
                "house_material": c.get("house_material"),
                "dist_km": c.get("distance_km"),
                "metro": c.get("metro_station"),
            }
        )
    )
```

```
payload = {
    "object": {
        "city": obj.get("city"),
        "address": obj.get("address"),
        "rooms": obj.get("rooms"),
        "area_total": obj.get("area_total"),
        "area_living": obj.get("area_living"),
        "area_kitchen": obj.get("area_kitchen"),
        "floor": obj.get("floor"),
        "floors_total": obj.get("floors_total"),
        "year_built": obj.get("year_built"),
        "house_material": obj.get("house_material"),
        "condition": obj.get("condition"),
    },
    "geo": {
        "lat": enr.get("lat"),
        "lon": enr.get("lon"),
        "dist_to_center_km": enr.get("dist_to_center_km"),
        "metro_station": enr.get("metro_station"),
        "dist_to_metro_m": enr.get("dist_to_metro_m"),
        "metro_walk_min": enr.get("metro_walk_min"),
        "density_500m": enr.get("density_500m"),
    },
    "pricing": pricing,
    "explanation_top_features": expl.get("top_features", []),
    "comparables": compact_comps,
}

toon_str = toon_encode(
    payload,
    {
        "indent": 2,
        "delimiter": ",",
        "lengthMarker": "#",
    },
)

return toon_str
```

Продолжение листинга B.1

```
_SYSTEM_PROMPT = (
    "Ты — эксперт по рынку жилой недвижимости Москвы. "
    "Твоя задача — кратко и по делу объяснить оценку рыночной месячной аренды
квартиры.\n"
    "Пиши по-русски, деловым, но понятным языком. "
    "Не используй сложных юридических формулировок, избегай канцелярита.\n"
    "Не упоминай, что ты ИИ или языковая модель. Не ссылайся на обучающие
данные.\n"
)

def _build_prompt(report: Dict[str, Any]) -> str:
    toon_block = _build_toon_payload(report)

    pricing = report.get("pricing", {}) or {}
    pred = pricing.get("prediction_rub")
    lo = pricing.get("interval_low_rub")
    hi = pricing.get("interval_high_rub")

    # аккуратно округлим до тысяч, чтобы текст был красивее
    def _round_thousand(x):
        if x is None:
            return None
        return int(round(x / 1000.0) * 1000)

    pred_r = _round_thousand(pred)
    lo_r = _round_thousand(lo)
    hi_r = _round_thousand(hi)

    # сделаем текстовый факт-блок, который модель должна уважать
    price_facts = []
    if pred_r is not None:
        price_facts.append(f"точечная оценка модели: {pred_r} ₽ в месяц")
    if lo_r is not None and hi_r is not None:
        price_facts.append(
            f"интервал значений: от {lo_r} ₽ до {hi_r} ₽ в месяц"
        )
```

Продолжение листинга B.1

```
price_facts_str = "; ".join(price_facts) if price_facts else "нет доступных  
числовых оценок."
```

```
prompt = f""">{_SYSTEM_PROMPT}
```

Даны данные об объекте и расчёте в формате TOON:

```
{toon_block}
```

Ключевые числовые значения ОКОНЧАТЕЛЬНОЙ оценки аренды (раздел pricing):

```
- {price_facts_str}
```

ОЧЕНЬ ВАЖНО:

- Все числовые оценки стоимости аренды (конкретная цена и диапазон) ты обязан брать ТОЛЬКО из раздела pricing.
- НЕЛЬЗЯ использовать цены сопоставимых объектов (comparables) как диапазон итоговой оценки.
- НЕЛЬЗЯ придумывать свои новые минимумы/максимумы цены.
- Допускается лёгкое округление до тысяч (например, 165 600 → 166 000), но границы и масштаб должны соответствовать указанным значениям.

Сформируй JSON с тремя полями:

- "summary_short" – 1-2 предложения для краткого блока на сайте. В этом блоке обязательно упомяни конечную оценку и (если есть) диапазон, используя только числа из pricing.
- "summary_long" – 2-4 абзаца подробного объяснения, можно ссылаться на те же числа, а также описывать факторы (район, площадь, метро и т.п.).
- "factors_summary" – массив из 3-7 коротких маркеров (строк), каждый – один важный фактор, влияющий на цену (например: "Большая площадь квартиры", "Близость к метро", "Современный дом" и т.п.).

Ответ верни ТОЛЬКО в виде JSON-объекта без дополнительных комментариев и Markdown.

```
"""
```

```
return prompt
```

Продолжение листинга В.1

```
def generate_text_blocks(report: Dict[str, Any], cfg: Optional[LLMConfig] =
None) -> Dict[str, Any]:
    cfg = cfg or LLMConfig()
    pricing = report.get("pricing", {}) or {}
    obj = report.get("object", {}) or {}

    pred = pricing.get("prediction_rub")
    lo = pricing.get("interval_low_rub")
    hi = pricing.get("interval_high_rub")

    def _fmt(x):
        if x is None:
            return None
        return f"{int(round(x)):,}".replace(",", " ")

    # детерминированный summary_short
    parts = []
    city = obj.get("city")
    addr = obj.get("address")
    if city:
        parts.append(f"в городе {city}")
    if addr:
        parts.append(f"по адресу «{addr}»")

    header = "Квартира"
    if parts:
        header += " " + ", ".join(parts)

    if pred is not None and lo is not None and hi is not None:
        summary_short = (
            f"{header}. Оценочная ставка долгосрочной аренды: около
{_fmt(pred)} ₽ в месяц "
            f"(ожидаемый диапазон { _fmt(lo) }-{ _fmt(hi) } ₽)."
        )
    elif pred is not None:
        summary_short = (
            f"{header}. Оценочная ставка долгосрочной аренды: около
{_fmt(pred)} ₽ в месяц."
        )
```

```
else:
    summary_short = f"{header}. Оценочная ставка не может быть рассчитана."

# если LLM выключен – вернём только короткое резюме
if not cfg.enabled:
    return {
        "summary_short": summary_short,
        "summary_long": summary_short,
        "factors_summary": [],
    }

# иначе просим LLM сгенерировать только long + факторы
llm = _get_llm(cfg)
prompt = _build_prompt(report) # обновлённый, как выше
resp = llm.invoke(prompt)
text = getattr(resp, "content", resp)
if not isinstance(text, str):
    text = str(text)
try:
    data = json.loads(text)
except json.JSONDecodeError:
    return {
        "summary_short": summary_short,
        "summary_long": text.strip(),
        "factors_summary": [],
    }

long_txt = data.get("summary_long", "") or ""
factors = data.get("factors_summary") or []
if isinstance(factors, str):
    factors_list = [factors]
elif isinstance(factors, list):
    factors_list = [str(x) for x in factors]
else:
    factors_list = [str(factors)]
return {
    "summary_short": summary_short,
    "summary_long": long_txt.strip() or summary_short,
    "factors_summary": [f.strip() for f in factors_list if str(f).strip()],
}
```


Продолжение листинга В.1

```
def _haversine_km(lat1, lon1, lat2, lon2):
    """Расчет расстояния между двумя точками в км"""
    R = 6371.0
    p = np.deg2rad(lat2 - lat1)
    l = np.deg2rad(lon2 - lon1)
    a = np.sin(p/2)**2 +
np.cos(np.deg2rad(lat1))*np.cos(np.deg2rad(lat2))*np.sin(l/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))

_ADDR_SPACE_RE = re.compile(r"\s+")

def _canon_addr(s: Optional[str]) -> str:
    """Упрощённая нормализация адреса для ключа кэша."""
    if not s:
        return ""
    s = s.replace("ё", "е").strip().lower()
    s = _ADDR_SPACE_RE.sub(" ", s)
    return s

def _rooms_bucket(r) -> str:
    """Категоризация квартир (студия, 1-комн, 2-комн и т.д.)"""
    try:
        r = int(r)
    except Exception:
        return "other"
    if r <= 0:
        return "studio"
    if r == 1:
        return "r1"
    if r == 2:
        return "r2"
    if r == 3:
        return "r3"
    if r == 4:
        return "r4"
    return "r5p"

def _walk_minutes_from_meters(meters: float, speed_m_per_min: float = 80.0) ->
int:
    """Конвертация метров в минуты пешком (скорость ~80 м/мин)"""
    if meters is None or np.isnan(meters):
        return None
    return int(math.ceil(meters / speed_m_per_min))

@retry(wait=wait_fixed(1), stop=stop_after_attempt(3))
def _nominatim_geocode(address: str, city: str = "Москва") ->
Optional[Tuple[float, float]]:
    """Геокодирование адреса через OSM Nominatim"""
    headers = {
        "User-Agent": "realval/0.1 (+grigorogannisyan.12@gmail.com)",
        "Accept-Language": "ru",
    }
    url = "https://nominatim.openstreetmap.org/search"
    params = {
        "street": address,
        "city": city,
        "countrycodes": "ru",
        "format": "jsonv2",
        "limit": 5,
    }
```

Продолжение листинга B.1

```
r = requests.get(url, params=params, headers=headers, timeout=20)
r.raise_for_status()
return float(data[0]["lat"]), float(data[0]["lon"])

def _density_500m(engine, lat: float, lon: float, days_back: int = 90) ->
Optional[int]:
    """Подсчет активных объявлений в радиусе 500м за последние N дней"""
    sql = """
SELECT COUNT(*) AS cnt
FROM listing_master
WHERE is_active AND last_seen >= NOW() - ( %(days)s || ' days')::interval
AND ST_DWithin(
    ST_SetSRID(ST_MakePoint( %(lon)s, %(lat)s), 4326)::geography,
    geom::geography,
    500)
    """
```

Листинга B.2 – Backend-шлюз и REST API

```
func main() {
    _ = godotenv.Load()
    isDev := os.Getenv("ENV") == "development"
    if err := logger.Init(isDev); err != nil {
        panic(err)
    }
    defer logger.Sync()
    log := logger.L()

    cfg := config.Load(log)

    db := database.ConnectDB(cfg, log)
    defer database.CloseDB(db, log)

    db.AutoMigrate(&models.ValuationReport{})

    repo := repository.NewValuationReportRepository(db)

    // Инициализация геокодера (Yandex или Nominatim)
    var geocoderClient geocode.GeocoderClient
    if cfg.YandexGeoAPI != "" {
        geocoderClient = geocode.NewYandexClient(cfg.YandexGeoAPI)
        log.Info("Yandex Geocoder client initialized")
    } else {
        geocoderClient = geocode.NewNominatimClient()
        log.Info("Nominatim (OpenStreetMap) Geocoder client initialized
(free alternative)")
    }

    // Инициализация Redis (опционально)
    var redisClient *cache.RedisClient
    if cfg.RedisAddr != "" {
        var err error
        redisClient, err = cache.NewRedisClient(
            cfg.RedisAddr,
            cfg.RedisPassword,
            cfg.RedisDB,
            cfg.RedisCachePrefix,
        )
        if err != nil {
            log.Warn("Failed to connect to Redis, caching disabled",
```

```

        zap.Error(err),
    )
    } else {
        log.Info("Redis client initialized")
        defer redisClient.Close()
    }
} else {
    log.Info("Redis not configured, caching disabled")
}

// Создаем geocode handler
geocodeHandler := handlers.NewGeocodeHandler(geocoderClient, redisClient)
r := httpapi.SetupRouter(cfg, repo, geocodeHandler, db, log)
port := ":" + cfg.Port
if err := r.Run(port); err != nil {
    log.Fatal("failed to run http server", zap.Error(err))
}
}

func SetupRouter(cfg *config.Config, repo repository.ValuationReportRepository,
geocodeHandler *handlers.GeocodeHandler, db *gorm.DB, log *zap.Logger)
*gin.Engine {
    r := gin.Default()

    r.Use(cors.New(cors.Config{
        AllowOrigins: []string{"*"},
        AllowMethods: []string{"GET", "POST", "PUT", "DELETE",
"OPTIONS"},
        AllowHeaders: []string{"Authorization", "Content-Type", "X-
Admin-Key"},
        ExposeHeaders: []string{"Content-Length"},
        AllowCredentials: true,
    })))
    r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))
    r.GET("/health", func(c *gin.Context) {
        c.String(200, "ok")
    })
    ml := mlclient.New(cfg.MLServiceURL,
time.Duration(cfg.MLTimeoutSec)*time.Second)
    valuationHandler := handlers.NewValuationHandler(ml, repo)

    api := r.Group("/api/v1")
    {
        valuation := api.Group("/valuation")
        {
            valuation.POST("/address", valuationHandler.PredictAddress)
            valuation.GET("/ml-health", valuationHandler.CheckMLHealth)
            valuation.GET("", valuationHandler.ListValuations)
            valuation.GET("/:id", valuationHandler.GetValuationByID)
            valuation.GET("/:id/pdf", valuationHandler.GetValuationPDF)
        }
        geocode := api.Group("/geocode")
        {
            geocode.GET("/suggest", geocodeHandler.SuggestAddress)
        }
    }
    return r
}

```

Продолжение листинга В.2

```
// Health проверяет доступность ML-сервиса
func (c *Client) Health(ctx context.Context) (*HealthResponse, error) {
    url := c.baseURL + "/health"

    httpReq, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return nil, fmt.Errorf("build request: %w", err)
    }

    resp, err := c.http.Do(httpReq)
    if err != nil {
        return nil, fmt.Errorf("call ML service health: %w", err)
    }
    defer resp.Body.Close()

    respBody, err := io.ReadAll(resp.Body)
    if err != nil {
        return nil, fmt.Errorf("read response: %w", err)
    }

    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("ML service health status %d: %s",
resp.StatusCode, string(respBody))
    }

    var health HealthResponse
    if err := json.Unmarshal(respBody, &health); err != nil {
        return nil, fmt.Errorf("decode health response: %w", err)
    }

    return &health, nil
}

func (c *Client) PredictAddress(ctx context.Context, req PredictAddressRequest)
(*Report, error) {
    url := c.baseURL + "/v1/predict/address"

    body, err := json.Marshal(req)
    if err != nil {
        return nil, fmt.Errorf("marshal request: %w", err)
    }

    httpReq, err := http.NewRequestWithContext(ctx, http.MethodPost, url,
bytes.NewReader(body))
    if err != nil {
        return nil, fmt.Errorf("build request: %w", err)
    }
    httpReq.Header.Set("Content-Type", "application/json")

    resp, err := c.http.Do(httpReq)
    if err != nil {
        return nil, fmt.Errorf("call ML service: %w", err)
    }
    defer resp.Body.Close()

    respBody, err := io.ReadAll(resp.Body)
    if err != nil {
        return nil, fmt.Errorf("read response: %w", err)
    }
}
```

```
    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("ML service status %d: %s", resp.StatusCode,
string(respBody))
    }

    var report Report
    if err := json.Unmarshal(respBody, &report); err != nil {
        return nil, fmt.Errorf("decode report: %w", err)
    }

    return &report, nil
}

func (h *ValuationHandler) PredictAddress(c *gin.Context) {
    var req AddressValuationRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, ErrorResponse{Error: err.Error()})
        return
    }

    if req.Rooms == nil {
        c.JSON(http.StatusBadRequest, ErrorResponse{Error: "rooms is
required"})
        return
    }

    mlReq := mlclient.PredictAddressRequest{
        City:      req.City,
        Address:    req.Address,
        Rooms:      *req.Rooms,
        AreaTotal:  req.AreaTotal,
        AreaLiving: req.AreaLiving,
        AreaKitchen: req.AreaKitchen,
        Floor:      req.Floor,
        FloorsTotal: req.FloorsTotal,
        YearBuilt:  req.YearBuilt,
        HouseMaterial: req.HouseMaterial,
        Condition:  req.Condition,
        WithText:   req.WithText,
    }

    ctx, cancel := context.WithTimeout(c.Request.Context(), 120*time.Second)
    defer cancel()

    report, err := h.ml.PredictAddress(ctx, mlReq)
    if err != nil {
        c.JSON(http.StatusBadGateway, ErrorResponse{Error: err.Error()})
        return
    }

    // маршалим полный отчёт в JSON для хранения
    raw, err := json.Marshal(report)
    if err != nil {
        c.JSON(http.StatusBadGateway, ErrorResponse{Error: "failed to
marshal ML report"})
        return
    }

    vr := &models.ValuationReport{
        // ID заполнит репозиторий (uuid.New())
    }
```

```

        City:          report.Object.City,
        Address:       report.Object.Address,
        DealType:      report.Pricing.DealType,
        PredictionRub: report.Pricing.PredictionRub,
        IntervalLowRub: report.Pricing.IntervalLowRub,
        IntervalHighRub: report.Pricing.IntervalHighRub,
        RawReport:     raw,
    }
    if err := h.repo.Save(ctx, vr); err != nil {
        c.JSON(http.StatusBadGateway, ErrorResponse{Error: err.Error()})
        return
    }

    report.ReportID = vr.ID.String()

    thin := mapReportToAddressResponse(report)
    c.JSON(http.StatusOK, thin)
}

func mapReportToAddressResponse(r *mlclient.Report) AddressValuationResponse {
    obj := AddressValuationObjectSummary{
        City:          r.Object.City,
        Address:       r.Object.Address,
        Rooms:         r.Object.Rooms,
        AreaTotal:     r.Object.AreaTotal,
        Floor:         r.Object.Floor,
        FloorsTotal:   r.Object.FloorsTotal,
        YearBuilt:     r.Object.YearBuilt,
        HouseMaterial: r.Object.HouseMaterial,
    }

    price := AddressValuationPriceSummary{
        PredictionRub:   r.Pricing.PredictionRub,
        IntervalLowRub:  r.Pricing.IntervalLowRub,
        IntervalHighRub: r.Pricing.IntervalHighRub,
        Currency:        r.Pricing.Currency,
        DealType:        r.Pricing.DealType,
    }

    var text *AddressValuationTextSummary
    if r.Text != nil {
        text = &AddressValuationTextSummary{
            SummaryShort:  r.Text.SummaryShort,
            SummaryLong:  r.Text.SummaryLong,
            FactorsSummary: append([]string{}, r.Text.FactorsSummary...),
        }
    }

    comparables := make([]AddressValuationComparableSummary, 0, 3)
    for i, c := range r.Comparables {
        if i >= 3 {
            break
        }
        comparables = append(comparables,
AddressValuationComparableSummary{
            PriceRub:      c.PriceRub,
            Rooms:        c.Rooms,
            AreaTotal:    c.AreaTotal,
            DistanceKm:   c.DistanceKm,

```

```

        MetroStation: c.MetroStation,
        URL:          c.URL, // <-- ДОБАВЛЕНО
    })
}
return AddressValuationResponse{
    ReportID:    r.ReportID,
    Object:      obj,
    Price:       price,
    Text:        text,
    Comparables: comparables,
}
}

func (h *ValuationHandler) GetValuationPDF(c *gin.Context) {
    idStr := c.Param("id")
    id, err := uuid.Parse(idStr)
    if err != nil {
        c.JSON(http.StatusBadRequest, ErrorResponse{Error: "invalid id"})
        return
    }

    ctx, cancel := context.WithTimeout(c.Request.Context(), 10*time.Second)
    defer cancel()

    vr, err := h.repo.GetByID(ctx, id)
    if err != nil {
        c.JSON(http.StatusInternalServerError, ErrorResponse{Error:
err.Error()})
        return
    }
    if vr == nil {
        c.JSON(http.StatusNotFound, ErrorResponse{Error: "report not
found"})
        return
    }

    var full mlclient.Report
    if err := json.Unmarshal(vr.RawReport, &full); err != nil {
        c.JSON(http.StatusInternalServerError, ErrorResponse{Error: "failed
to decode stored report"})
        return
    }
    full.ReportID = vr.ID.String()

    pdfBytes, err := reportpdf.RenderReportPDF(&full, reportpdf.Options{})
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{
            "error": "failed to render pdf",
            "detail": err.Error(), // <-- покажет реальную ошибку
        })
        return
    }

    filename := fmt.Sprintf("report_%s.pdf", vr.ID.String())
    c.Header("Content-Type", "application/pdf")
    c.Header("Content-Disposition", fmt.Sprintf("attachment; filename=%q",
filename))
    c.Data(http.StatusOK, "application/pdf", pdfBytes)
}

```

```
        MetroStation: c.MetroStation,
        URL:          c.URL, // <-- ДОБАВЛЕНО
    })
}
return AddressValuationResponse{
    ReportID:    r.ReportID,
    Object:      obj,
    Price:       price,
    Text:        text,
    Comparables: comparables,
}
}

func (h *ValuationHandler) GetValuationPDF(c *gin.Context) {
    idStr := c.Param("id")
    id, err := uuid.Parse(idStr)
    if err != nil {
        c.JSON(http.StatusBadRequest, ErrorResponse{Error: "invalid id"})
        return
    }
    ctx, cancel := context.WithTimeout(c.Request.Context(), 10*time.Second)
    defer cancel()

    vr, err := h.repo.GetByID(ctx, id)
    if err != nil {
        c.JSON(http.StatusInternalServerError, ErrorResponse{Error:
err.Error()})
        return
    }
    if vr == nil {
        c.JSON(http.StatusNotFound, ErrorResponse{Error: "report not
found"})
        return
    }

    var full mlclient.Report
    if err := json.Unmarshal(vr.RawReport, &full); err != nil {
        c.JSON(http.StatusInternalServerError, ErrorResponse{Error: "failed
to decode stored report"})
        return
    }
    full.ReportID = vr.ID.String()

    pdfBytes, err := reportpdf.RenderReportPDF(&full, reportpdf.Options{})
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{
            "error": "failed to render pdf",
            "detail": err.Error(), // <-- покажет реальную ошибку
        })
        return
    }

    filename := fmt.Sprintf("report_%s.pdf", vr.ID.String())
    c.Header("Content-Type", "application/pdf")
    c.Header("Content-Disposition", fmt.Sprintf("attachment; filename=%q",
filename))
    c.Data(http.StatusOK, "application/pdf", pdfBytes)
}
```



```
// NewNominatimClient создает новый клиент для Nominatim
func NewNominatimClient() *NominatimClient {
    return &NominatimClient{
        httpClient: &http.Client{
            Timeout: 10 * time.Second,
        },
        baseURL:    "https://nominatim.openstreetmap.org/search",
        userAgent:  "RentAdvisor/1.0", // Обязательно для Nominatim
    }
}

// cleanAddressNominatim удаляет префикс из адреса Nominatim
func cleanAddressNominatim(address string) string {
    prefixes := []string{
        "Россия, Москва, ",
        "Россия, Москва,",
        "Москва, Россия, ",
        "Москва, Россия,",
        "Москва, ",
    }
    for _, prefix := range prefixes {
        if len(address) > len(prefix) && address[:len(prefix)] == prefix {
            return address[len(prefix):]
        }
    }
    return address
}

// SuggestAddresses ищет адреса по запросу через Nominatim
func (c *NominatimClient) SuggestAddresses(ctx context.Context, query string,
limit int) ([]AddressSuggestion, error) {
    if query == "" {
        return []AddressSuggestion{}, nil
    }

    if limit <= 0 {
        limit = 5
    }

    // Добавляем "Москва" для более точного поиска
    searchQuery := query + ", Москва, Россия"

    params := url.Values{}
    params.Set("q", searchQuery)
    params.Set("format", "json")
    params.Set("addressdetails", "1")
    params.Set("limit", fmt.Sprintf("%d", limit))
    params.Set("countrycodes", "ru")

    reqURL := c.baseURL + "?" + params.Encode()

    req, err := http.NewRequestWithContext(ctx, "GET", reqURL, nil)
    if err != nil {
        return nil, fmt.Errorf("failed to create request: %w", err)
    }

    req.Header.Set("User-Agent", c.userAgent)

    resp, err := c.httpClient.Do(req)
```

```
        if err != nil {
            return nil, fmt.Errorf("failed to make request: %w", err)
        }
        defer resp.Body.Close()

        body, err := io.ReadAll(resp.Body)
        if err != nil {
            return nil, fmt.Errorf("failed to read response: %w", err)
        }

        if resp.StatusCode != http.StatusOK {
            return nil, fmt.Errorf("nominatim returned status %d: %s",
resp.StatusCode, string(body))
        }

        var nomResp NominatimResponse
        if err := json.Unmarshal(body, &nomResp); err != nil {
            return nil, fmt.Errorf("failed to parse response: %w", err)
        }

        // Конвертируем в наш формат
        suggestions := make([]AddressSuggestion, 0, len(nomResp))

        for _, item := range nomResp {
            // Парсим координаты
            var lat, lon float64
            fmt.Sscanf(item.Lat, "%f", &lat)
            fmt.Sscanf(item.Lon, "%f", &lon)

            // Формируем описание
            description := ""
            if item.Address.Suburb != "" {
                description = item.Address.Suburb
            } else if item.Address.Municipality != "" {
                description = item.Address.Municipality
            }

            suggestion := AddressSuggestion{
                Address:      cleanAddressNominatim(item.DisplayName),
                Description: description,
                Latitude:      lat,
                Longitude:   lon,
            }

            suggestions = append(suggestions, suggestion)
        }

        return suggestions, nil
    }

// newYandexClient создает новый клиент для Yandex Geocoder
func newYandexClient(apiKey string) *YandexClient {
    return &YandexClient{
        apiKey: apiKey,
        httpClient: &http.Client{
            Timeout: 10 * time.Second,
        },
        baseURL: "https://geocode-maps.yandex.ru/1.x/",
    }
}
```

```
}

// SuggestAddresses ищет адреса по запросу
func (c *YandexClient) SuggestAddresses(ctx context.Context, query string, limit
int) ([]AddressSuggestion, error) {
    if query == "" {
        return []AddressSuggestion{}, nil
    }

    if limit <= 0 {
        limit = 5
    }

    // Добавляем "Москва, Россия" для более точного поиска
    searchQuery := fmt.Sprintf("%s, Москва, Россия", query)

    // Строим URL с параметрами
    params := url.Values{}
    params.Set("apikey", c.apiKey)
    params.Set("geocode", searchQuery)
    params.Set("format", "json")
    params.Set("results", fmt.Sprintf("%d", limit))
    params.Set("kind", "house") // Фокус на домах, а не районах

    reqURL := c.baseURL + "?" + params.Encode()

    req, err := http.NewRequestWithContext(ctx, "GET", reqURL, nil)
    if err != nil {
        return nil, fmt.Errorf("failed to create request: %w", err)
    }

    resp, err := c.httpClient.Do(req)
    if err != nil {
        return nil, fmt.Errorf("failed to make request: %w", err)
    }
    defer resp.Body.Close()

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return nil, fmt.Errorf("failed to read response: %w", err)
    }

    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("yandex geocoder returned status %d: %s",
resp.StatusCode, string(body))
    }

    var geoResp YandexGeocoderResponse
    if err := json.Unmarshal(body, &geoResp); err != nil {
        return nil, fmt.Errorf("failed to parse response: %w, body: %s",
err, string(body))
    }

    // Конвертируем в наш формат
    suggestions := make([]AddressSuggestion, 0,
len(geoResp.Response.GeoObjectCollection.FeatureMember))

    for _, member := range geoResp.Response.GeoObjectCollection.FeatureMember
{
```

```

        geoObj := member.GeoObject
        metaData := geoObj.MetaDataProperty.GeocoderMetaData

        // Парсим координаты (формат: "longitude latitude")
        var lon, lat float64
        fmt.Sscanf(geoObj.Point.Pos, "%f %f", &lon, &lat)

        suggestion := AddressSuggestion{
            Address:      cleanAddress(metaData.Address.Formatted),
            Description:   geoObj.Description,
            Latitude:      lat,
            Longitude:     lon,
        }

        suggestions = append(suggestions, suggestion)
    }

    return suggestions, nil
}

func RenderReportPDF(r *mlclient.Report, opts Options) ([]byte, error) {
    pdf, err := newPDFWithFont(opts)
    if err != nil {
        return nil, err
    }

    // ---- Заголовок ----
    title := "Отчет об оценке арендной стоимости"
    pdf.CellFormat(190, 10, title, "", 1, "C", false, 0, "")

    pdf.Ln(2)
    pdf.SetFont("timesnewromanpsmt", "", 11)
    addrLine := fmt.Sprintf("%s, %s", r.Object.City, r.Object.Address)
    pdf.MultiCell(190, 6, addrLine, "", "C", false)

    pdf.Ln(4)

    // ---- 1. Итоговая оценка ----
    pdf.SetFont("timesnewromanpsmt", "B", 13)
    pdf.CellFormat(190, 8, "1. Итоговая оценка аренды", "", 1, "", false, 0,
    "")
    pdf.SetFont("timesnewromanpsmt", "", 11)

    mainText := fmt.Sprintf(
        "Оценочная ставка долгосрочной аренды: %.0f руб. в месяц.\n"+
        "Ожидаемый диапазон: от %.0f до %.0f руб. в месяц.\n"+
        "Валюта: %s, тип сделки: %s.",
        r.Pricing.PredictionRub,
        r.Pricing.IntervalLowRub,
        r.Pricing.IntervalHighRub,
        r.Pricing.Currency,
        r.Pricing.DealType,
    )
    pdf.MultiCell(190, 6, mainText, "", "", false)
    pdf.Ln(3)

    // ---- 2. Характеристики объекта ----
    pdf.SetFont("timesnewromanpsmt", "B", 13)
    pdf.CellFormat(190, 8, "2. Характеристики объекта", "", 1, "", false, 0,

```

```

    "")
    pdf.SetFont("timesnewromanpsmt", "", 11)

    objLines := []string{
        fmt.Sprintf("Город: %s", r.Object.City),
        fmt.Sprintf("Адрес: %s", r.Object.Address),
    }

    // Определяем тип жилья
    if r.Object.Rooms == 0 {
        objLines = append(objLines, "Тип: студия")
    } else {
        objLines = append(objLines, fmt.Sprintf("Комнат: %d",
r.Object.Rooms))
    }

    objLines = append(objLines, fmt.Sprintf("Общая площадь: %.1f м²",
r.Object.AreaTotal))

    if r.Object.Floor != nil && r.Object.FloorsTotal != nil {
        objLines = append(objLines, fmt.Sprintf("Этаж: %d из %d",
*r.Object.Floor, *r.Object.FloorsTotal))
    }
    if r.Object.YearBuilt != nil {
        objLines = append(objLines, fmt.Sprintf("Год постройки: %d",
*r.Object.YearBuilt))
    }
    if r.Object.HouseMaterial != nil && *r.Object.HouseMaterial != "" {
        objLines = append(objLines, fmt.Sprintf("Материал дома: %s",
*r.Object.HouseMaterial))
    }
    if r.Object.Condition != nil && *r.Object.Condition != "" {
        objLines = append(objLines, fmt.Sprintf("Состояние: %s",
*r.Object.Condition))
    }

    for _, line := range objLines {
        pdf.CellFormat(190, 6, line, "", 1, "", false, 0, "")
    }
    pdf.Ln(3)

    // ---- 3. Геоположение и окружение ----
    pdf.SetFont("timesnewromanpsmt", "B", 13) // <-- ИЗМЕНЕНО
    pdf.CellFormat(190, 8, "3. Геоположение и окружение", "", 1, "", false,
0, "")
    pdf.SetFont("timesnewromanpsmt", "", 11) // <-- ИЗМЕНЕНО

    if r.Enriched.DistToCenterKm != nil {
        pdf.CellFormat(190, 6,
            fmt.Sprintf("Расстояние до центра города: %.1f км",
*r.Enriched.DistToCenterKm),
            "", 1, "", false, 0, "")
    }
    if r.Enriched.MetroStation != nil && *r.Enriched.MetroStation != "" {
        line := fmt.Sprintf("Ближайшее метро: %s",
*r.Enriched.MetroStation)
        if r.Enriched.MetroWalkMin != nil {
            mins := int(math.Round(*r.Enriched.MetroWalkMin))
            line += fmt.Sprintf(", ~%d минут пешком", mins)
        }
    }

```

```

    }
    pdf.CellFormat(190, 6, line, "", 1, "", false, 0, "")
}
if r.Enriched.Density500m != nil {
    pdf.CellFormat(190, 6,
        fmt.Sprintf("Плотность предложений в радиусе 500 м (по
выборке): %.0f", *r.Enriched.Density500m),
        "", 1, "", false, 0, "")
}
pdf.Ln(3)

// ---- 4. Текстовое объяснение (LLM) ----
if r.Text != nil {
    pdf.SetFont("timesnewromanpsmt", "B", 13) // <-- ИЗМЕНЕНО
    pdf.CellFormat(190, 8, "4. Объяснение оценки", "", 1, "", false, 0,
    "")

    pdf.SetFont("timesnewromanpsmt", "", 11) // <-- ИЗМЕНЕНО

    pdf.MultiCell(190, 5, r.Text.SummaryLong, "", "", false)
    pdf.Ln(2)

    if len(r.Text.FactorsSummary) > 0 {
        pdf.CellFormat(190, 6, "Ключевые факторы:", "", 1, "", false,
0, "")

        for _, f := range r.Text.FactorsSummary {
            pdf.CellFormat(5, 5, "•", "", 0, "", false, 0, "")
            pdf.CellFormat(185, 5, f, "", 1, "", false, 0, "")
        }
    }
    pdf.Ln(3)
}

// ---- 5. Сопоставимые объекты ----
if len(r.Comparables) > 0 {
    pdf.SetFont("timesnewromanpsmt", "B", 13)
    pdf.CellFormat(190, 8, "5. Сопоставимые объекты", "", 1, "", false,
0, "")

    pdf.SetFont("timesnewromanpsmt", "", 10)

    // Заголовок таблицы
    pdf.SetFillColor(230, 230, 230)
    pdf.CellFormat(45, 7, "Цена, руб.", "1", 0, "C", true, 0, "")
    pdf.CellFormat(20, 7, "Комнат", "1", 0, "C", true, 0, "")
    pdf.CellFormat(30, 7, "Площадь, м²", "1", 0, "C", true, 0, "")
    pdf.CellFormat(35, 7, "Расстояние, км", "1", 0, "C", true, 0, "")
    pdf.CellFormat(60, 7, "Метро", "1", 1, "C", true, 0, "")

    pdf.SetFillColor(255, 255, 255)
    maxRows := 5
    for i, c := range r.Comparables {
        if i >= maxRows {
            break
        }
        price := fmt.Sprintf("%.0f", c.PriceRub)

        var rooms string
        if c.Rooms != nil {
            rooms = fmt.Sprintf("%d", *c.Rooms)
        }
    }
}

```

```

        var area string
        if c.AreaTotal != nil {
            area = fmt.Sprintf("%.1f", *c.AreaTotal)
        }
        var dist string
        if c.DistanceKm != nil {
            dist = fmt.Sprintf("%.2f", *c.DistanceKm)
        }
        var metro string
        if c.MetroStation != nil {
            metro = *c.MetroStation
        }
        pdf.CellFormat(45, 6, price, "1", 0, "C", false, 0, "")
        pdf.CellFormat(20, 6, rooms, "1", 0, "C", false, 0, "")
        pdf.CellFormat(30, 6, area, "1", 0, "C", false, 0, "")
        pdf.CellFormat(35, 6, dist, "1", 0, "C", false, 0, "")
        pdf.CellFormat(60, 6, metro, "1", 1, "C", false, 0, "")
        if c.URL != nil && *c.URL != "" {
            pdf.SetFont("timesnewromanpsmt", "", 9)
            pdf.SetTextColor(0, 0, 255) // синий цвет для ссылки
            displayURL := *c.URL
            if len(displayURL) > 60 {
                displayURL = displayURL[:57] + "..."
            }
            pdf.CellFormat(5, 5, "", "", 0, "", false, 0, "")
            pdf.WriteLinkString(5, displayURL, *c.URL)
            pdf.Ln(5)

            pdf.SetTextColor(0, 0, 0) // возвращаем черный цвет
            pdf.SetFont("timesnewromanpsmt", "", 10)
        }
    }
    pdf.Ln(3)
}

pdf.SetFont("timesnewromanpsmt", "B", 13) // <-- ИЗМЕНЕНО
pdf.CellFormat(190, 8, "6. Сведения о модели", "", 1, "", false, 0, "")
pdf.SetFont("timesnewromanpsmt", "", 11) // <-- ИЗМЕНЕНО

metricsLine := fmt.Sprintf(
    "Модель: %s, целевая переменная: %s, лог-преобразование: %v",
    r.ModelInfo.ModelName, r.ModelInfo.Target, r.ModelInfo.LogTarget,
)
pdf.CellFormat(190, 6, metricsLine, "", 1, "", false, 0, "")

if r.ModelInfo.ValidMAE != nil && r.ModelInfo.ValidRMSE != nil {
    pdf.CellFormat(190, 6,
        fmt.Sprintf("Качество на валидации: MAE=%.0f руб., RMSE=%.0f
руб.",
            *r.ModelInfo.ValidMAE, *r.ModelInfo.ValidRMSE),
        "", 1, "", false, 0, "")
    }

var buf bytes.Buffer
if err := pdf.Output(&buf); err != nil {
    return nil, err
}
return buf.Bytes(), nil
}

```